



AlphaPi Zero

开发者参考手册

ESP32-C3 + HT协控 · 5×5点阵编程教育硬件

版本 v1.0

2026-08-25

固件版本 2022102103

AlphaPi Zero 开发者参考手册

****设备型号****: ZERO (HW_NAME: ZERO_Oct 21)

****主控芯片****: ESP32-C3

****协控芯片****: HT 合泰 MCU

****固件版本****: 2022102103

****MicroPython****: 3.4.0 (2022-10-10 定制版)

****手册版本****: v1.0

****更新日期****: 2026-08-25

目录

- [第1章 快速上手](#第1章-快速上手)
- [第2章 硬件规格](#第2章-硬件规格)
- [第3章 开发环境](#第3章-开发环境)
- [第4章 protocol 模块 API 参考](#第4章-protocol-模块-api-参考)
- [第5章 协控通信协议详解](#第5章-协控通信协议详解)
- [第6章 5×5 LED 点阵编程指南](#第6章-55-led-点阵编程指南)
- [第7章 音频编程指南](#第7章-音频编程指南)
- [第8章 按键与输入指南](#第8章-按键与输入指南)
- [第9章 加速度计使用指南](#第9章-加速度计使用指南)
- [第10章 文件系统指南](#第10章-文件系统指南)
- [第11章 完整示例代码集](#第11章-完整示例代码集)
- [第12章 常见问题与故障排除](#第12章-常见问题与故障排除)
- [附录A 内置音效文件清单](#附录a-内置音效文件清单)
- [附录B 5×5 字模库](#附录b-55-字模库)

第1章 快速上手

1.1 设备简介

AlphaPi Zero (设备名 ZERO) 是一款基于 ESP32-C3 的迷你开发板, 配备 5×5 LED 点阵、3个按键、三轴加速度计、音频播放/录音功能。采用双芯片架构: ESP32-C3 运行 MicroPython 处理逻辑, HT 合泰 MCU 负责音频解码和点阵驱动。

1.2 第一次运行

连接设备到电脑, 打开串口终端 (如 Thonny、uPyCraft、PuTTY) , 波特率

115200, 按复位键后将看到:

```
MPY: soft reboot
HT protocol 1010
MicroPython e388c9f23 on 2022-10-10; ESP32C3 module with ESP32C3
Type "help()" for more information.
>>>
```

HT protocol 1010 表示协控通信初始化成功。

1.3 Hello World — 点亮点阵

```
import protocol as p

#
p.led_fill()

# 0
p.uart_write(8, bytes([0, 0, 0, 0, 0]))
```

1.4 播放音效

```
import protocol as p

p.play('alert.dat') #
```

1.5 读取按键

```
from machine import Pin
import time

key_a = Pin(10, Pin.IN, Pin.PULL_DOWN) # A
key_b = Pin(1, Pin.IN, Pin.PULL_DOWN) # B
key_c = Pin(3, Pin.IN, Pin.PULL_DOWN) # C

while True:
    if key_a.value() == 1:
        print("A")
    if key_b.value() == 1:
        print("B")
    if key_c.value() == 1:
        print("C")
    time.sleep_ms(50)
```

1.6 修改开机自启动

编辑设备上的 **main.py**, 添加你的代码:

```
import protocol as p
p.init_files()
p.check_hardware()

#
p.led_fill()
p.play('alert.dat')
```

第2章 硬件规格

2.1 主控芯片

参数	
	规格
型号	ESP32-C3
内核	RISC-V 单核 32位
主频	160 MHz
Flash	4MB
SRAM	400KB
WiFi	2.4 GHz 802.11 b/g/n
蓝牙	BLE 5.0
GPIO	22个（部分复用）

2.2 协控芯片

参数	
	规格
型号	HT 合泰 MCU（具体型号未标识）
职责	音频解码、5×5点阵驱动、录音
通信	UART，单向（ESP32→HT）

2.3 外设清单

外设		
	规格	接口
5×5 LED点阵	25颗LED，行列扫描	HT协控驱动
按键×3	A/B/C，轻触开关	ESP32 GPIO直连
加速度计	SC7A20，三轴，±2g	SoftI2C
扬声器	8Ω 0.5W	HT协控PWM/DA输出
麦克风	驻极体麦克风	HT协控ADC输入

2.4 引脚分配表

GPIO			
	功能	模式	说明
0	预留	-	可用GPIO
1	按键B	下拉输入	按下=高电平
2	预留	-	可用GPIO
3	按键C	下拉输入	按下=高电平
4	预留	-	可用GPIO
5	预留	-	可用GPIO
6	I2C SDA	开漏输出	加速度计数据线
7	I2C SCL	开漏输出	加速度计时钟线
8	UART1 TX	推挽输出	发往协控
9	UART1 RX	输入	协控发回（实际无数据）
10	按键A	下拉输入	按下=高电平
11	预留	-	可用GPIO
12	预留	-	可用GPIO
13	预留	-	可用GPIO
14-17	不可用	-	ESP32-C3内部使用
18	预留	-	可用GPIO
19	预留	-	可用GPIO
20	USB D+	-	USB调试串口
21	USB D-	-	USB调试串口

2.5 电气特性

参数			
	最小值	典型值	最大值
工作电压	3.0V	3.3V	3.6V
工作电流（运行）	-	80mA	150mA
工作电流（深度睡眠）	-	10μA	100μA
工作温度	-40°C	25°C	85°C

2.6 P1 扩展接口

主板提供 P1 扩展接口（4针JST），用于连接官方 RGB 灯板等扩展模块。

引脚		
	功能	说明
1	VCC	3.3V 电源输出
2	GND	地
3	DIN	扩展数据输入 (GPIO5, WS2812数据)
4	预留	未使用 (可用于级联 DOUT 或其他扩展)

注意： P1 接口供电能力有限，连接大电流外设（如大量WS2812）时建议外接电源。

2.7 RGB 灯板扩展

官方 RGB 灯板通过 P1 接口连接，详细编程指南见**附录D**。

参数	
	规格
LED型号	WS2812 (内置IC全彩LED)
LED数量	14颗 (单面, 位于灯板反面)
排列方式	菱形/环形, 编号1-14
数据引脚	GPIO5 (P1第3针)
颜色深度	24位 RGB, 1677万色
驱动方式	内置 neopixel 模块 (ESP32 RMT硬件时序)
供电	3.3V (P1接口提供)

注意： 灯板正面为装饰面（有编号丝印和贴片电阻），LED芯片均焊接在反面，使用时反面向外。

第3章 开发环境

3.1 推荐IDE

Thonny (推荐新手)

1. 下载安装 [Thonny](https://thonny.org/) 2. 运行后选择 → → **MicroPython (ESP32)** 3. 选择对应的串口 (COMx) 4. 即可在编辑器中编写代码，按运行按钮上传到设备

uPyCraft

专为 MicroPython 设计的IDE，支持文件管理、串口终端、固件烧录。

VS Code + Pymakr 插件

适合熟悉 VS Code 的开发者，提供语法高亮、自动补全、一键上传。

3.2 串口终端参数

参数

	值
波特率	115200
数据位	8
停止位	1
校验位	无
流控	无

3.3 文件操作

列出文件

```
import os
print(os.listdir())
```

读取文件

```
with open('main.py', 'r') as f:
    print(f.read())
```

写入文件

```
with open('main.py', 'w') as f:
    f.write('import protocol as p\np.led_fill()\n')
```

删除文件

```
import os
os.remove('test.py')
```

查看文件大小

```
import os
stat = os.stat('alert.dat')
print(":", stat[6], "")
```

3.4 REPL 快捷键

快捷键

	功能
`Ctrl+C`	中断当前运行程序
`Ctrl+D`	软复位（重新运行 boot.py + main.py）
`Ctrl+E`	进入粘贴模式（批量输入代码）
`Ctrl+A`	进入原始REPL模式

3.5 内存管理

```
import gc

#
print(":", gc.mem_alloc(), "")
print(":", gc.mem_free(), "")

#
gc.collect()
```

ESP32-C3 可用内存约 100KB+, 足够运行大多数应用。

3.6 固件恢复

如果设备变砖, 可通过 ESP32 串口烧录工具重新烧录 MicroPython 固件:

1. 下载 [MicroPython ESP32-C3 固件](https://micropython.org/download/esp32/) 2. 使用 `esptool.py` 烧录: `bash esptool.py --port COM3 --baud 460800 write_flash -z 0x0 firmware.bin` 3. protocal` 模块, 官方通用固件没有此模块`

第4章 protocal 模块 API 参考

`protocal` 是设备核心模块, 编译进固件 (frozen module), 提供所有硬件控制接口。

4.1 模块属性

属性	类型	说明
<code>`hal`</code>	module	硬件抽象层子模块
<code>`uart`</code>	UART	协控串口对象 (UART1)
<code>`machine`</code>	module	machine 模块引用
<code>`time`</code>	module	utime 模块引用
<code>`esp`</code>	module	esp 模块引用
<code>`gc`</code>	module	gc 模块引用
<code>`zlib`</code>	module	uzlib 模块引用

4.2 系统函数

``board_info()``

打印硬件版本信息, 无返回值。

```
import protocal as p
p.board_info()
# :
# SW_VER: 2022102103
# HW_NAME: ZERO_Oct 21
```

``init_files()``

初始化文件系统, 确保必要的系统文件存在。应在 `main.py` 开头调用。

```
import protocol as p
p.init_files()
```

``check硬件()``

硬件自检，会点亮所有LED（点阵全亮闪烁），验证硬件连接正常。

```
import protocol as p
p.check硬件()
```

``calc_checksum(data)``

计算数据校验和，用于协控协议。

参数

	类型	说明
<code>`data`</code>	bytes	待校验数据

返回：校验和（整数）

```
import protocol as p
cs = p.calc_checksum(b'\x08\xff\xff\xff\xff')
print(":", cs)
```

4.3 点阵控制函数

``led_fill()``

全屏点亮所有LED。无参数，无返回值。

```
import protocol as p
p.led_fill() #
```

等效协控指令：`uart_write(8, b'\xff\xff\xff\xff')`

``progress(n)``

显示进度条，按列从左到右点亮。

参数

	类型	范围	说明
<code>`n`</code>	int	0~255	进度值，映射到0~25个像素

```
import protocol as p
import time

for i in range(0, 26):
    p.progress(i)
    time.sleep(0.1)
```

显示逻辑：

- `progress(0)` → 全灭
- `progress(5)` → 第1列全亮 (5个像素)
- `progress(10)` → 前2列全亮 (10个像素)
- `progress(25)` → 全部25个像素全亮
- 超过25的值按取模或截断处理

4.4 音频函数

``play(filename)``

播放音频文件。支持标准WAV格式和原始PCM格式。

参数		类型	说明
<code>`filename`</code>	str		音频文件名 (如 'alert.dat')

```
import protocol as p
p.play('alert.dat') #
```

支持格式:

- WAV (带RIFF头) : 8位, 单声道或双声道, 22050Hz
- 原始PCM (无头) : 8位, 默认22050Hz单声道

注意:

- 函数是异步的, 调用后立即返回, 音频由协控后台播放
- 播放新音频会中断当前播放
- 文件名不需要完整路径, 直接使用根目录文件名

``rec(filename)``

录制音频并保存为文件。

参数		类型	说明
<code>`filename`</code>	str		保存的录音文件名

```
import protocol as p
p.rec('myrecording') # myrecording
```

录音参数:

- 格式: 原始8位PCM (无WAV头)
- 采样率: 22050Hz
- 固定大小: 61440 字节 (约2.78秒)
- 录音过程中会播放 `record_start.dat` 和 `record_end.dat` 提示音

回放录音:

```
p.play('myrecording') # PCM
```

转换为WAV（在电脑上操作）：

```
# 44WAV
import struct
with open('myrecording', 'rb') as f:
    pcm = f.read()
sample_rate = 22050
wav_header = struct.pack('<4sI4s4sIHIIHH4sI',
    b'RIFF', 36 + len(pcm), b'WAVE',
    b'fmt ', 16, 1, 1, sample_rate, sample_rate, 1, 8,
    b'data', len(pcm))
with open('myrecording.wav', 'wb') as f:
    f.write(wav_header + pcm)
```

4.5 串口通信函数

``uart_write(cmd, data)``

向协控发送指令。这是最底层的通信接口。

参数

类型

说明

<code>`cmd`</code>	int	指令码（十进制）
<code>`data`</code>	bytes/bytearray	数据负载

```
import protocol as p

#
p.uart_write(8, b'\xFF\xFF\xFF\xFF\xFF')

#
p.uart_write(20, b'\x04')

#
p.uart_write(22, audio_data)
```

已确认指令码：

指令码

功能

数据格式

8	点阵显示	5字节，每字节1列
16	录音控制	1字节：0x02=开始/查询, 0x00=停止
20	音频启动	1字节：0x04=播放模式
22	音频数据	WAV/PCM数据流，每包约256字节

``uart_read(timeout, length)``

从协控读取数据。**实际协控不返回数据**，此函数主要用于内部协议。

参数

	类型	说明
`timeout`	int	超时时间 (ms)
`length`	int	读取长度

返回: bytes对象

```
import protocol as p
data = p.uart_read(100, 64)
print(":", data)
```

`uart\_write\_buf(data)`

底层串口写缓冲区, 直接操作硬件串口发送数据。

```
import protocol as p
p.uart_write_buf(b'\x08\xff\xff\xff\xff\xff')
```

`uart\_read\_buf()`

底层串口读缓冲区, 直接从硬件串口读取数据。

```
import protocol as p
data = p.uart_read_buf()
```

4.6 文件操作函数

`readFile(name)`

读取文件内容。

参数

	类型	说明
`name`	str	文件名

返回: 文件内容 (bytes)

```
import protocol as p
data = p.readFile('main.py')
print(data)
```

`get\_files\_from\_flash()`

从flash获取文件列表。

```
import protocol as p
files = p.get_files_from_flash()
print(files)
```

4.7 固件升级函数

`update\_mcu`

协控MCU固件升级。用于升级HT合泰芯片的固件。

```
import protocol as p
p.update_mcu()
```

警告：此功能需要配合固件升级文件使用，错误操作可能导致协控芯片变砖，普通开发者请勿随意调用。

4.8 hal 子模块

`protocol.hal` 提供底层硬件抽象。

属性

属性		类型	说明
`key_map`	dict		按键映射：`{'A': Pin(10), 'B': Pin(1), 'C': Pin(3)}`
`i2c_map`	dict		I2C映射：`{'ACC': SoftI2C(scl=7, sda=6, freq=500000)}`
`uart_mcu`	UART		协控串口对象（同 `protocol.uart`）
`Pin`	type		Pin类引用
`SoftI2C`	type		SoftI2C类引用
`machine`	module		machine模块引用

使用示例

```
import protocol as p

# Pin
key_a = p.hal.key_map['A']
print("A:", key_a.value())

# I2C
i2c = p.hal.i2c_map['ACC']
print("I2C:", i2c.scan())

#
uart = p.hal.uart_mcu
print(":", uart)
```

第5章 协控通信协议详解

5.1 通信架构

ESP32-C3（主控）与 HT 合泰 MCU（协控）通过 UART1 通信，采用**单向指令模式**：

ESP32-C3 —UART1 TX(GPIO8)—> HTMCU
ESP32-C3 <—UART1 RX(GPIO9)— HTMCU

关键特性：

- 单向通信：ESP32发送指令，协控执行，不返回应答
- 无需握手：发送即执行，无需等待确认
- 实时控制：点阵、音频等指令实时生效

5.2 串口参数

参数	值
串口号	UART1
波特率	460929
数据位	8
停止位	1
校验位	无
流控	无
TX缓冲区	256字节
RX缓冲区	256字节
超时	100ms

5.3 指令格式

```
uart_write(cmd, data)
cmd = 1
data =
```

5.4 指令码总表

指令码	十六进制	功能	数据长度	说明
8	0x08	点阵显示	5字节	每字节对应一列
16	0x10	录音控制	1字节	0x02=开始, 0x00=停止
20	0x14	音频启动	1字节	0x04=播放模式

22	0x16	音频数据	≤256字节	WAV/PCM数据流
----	------	------	--------	------------

5.5 点阵显示指令（码8）

数据格式

```
0 = 1
1 = 2
2 = 3
3 = 4
4 = 5
```

每字节位定义

```
bit7 = 1
bit6 = 2
bit5 = 3
bit4 = 4
bit3 = 5
bit2 = 0
bit1 = 0
bit0 = 0
```

示例

```
import protocol as p

#
p.uart_write(8, bytes([0xFF, 0xFF, 0xFF, 0xFF, 0xFF]))

#
p.uart_write(8, bytes([0x00, 0x00, 0x00, 0x00, 0x00]))

# 1
p.uart_write(8, bytes([0xF8, 0x00, 0x00, 0x00, 0x00]))

# 5
p.uart_write(8, bytes([0x00, 0x00, 0x00, 0x00, 0xF8]))

# 33
p.uart_write(8, bytes([0x00, 0x00, 0x20, 0x00, 0x00]))
# 0x20 = 00100000bit5=1 → 3

#
heart_cols = [
    0x50, # 1: 01010000 → 24
    0xF8, # 2: 11111000 →
    0xF8, # 3:
    0x70, # 4: 01110000 → 234
    0x20, # 5: 00100000 → 3
]
p.uart_write(8, bytes(heart_cols))
```

5.6 音频播放指令（码20+22）

播放流程

```
1. uart_write(20, b'\x04') →  
2. uart_write(22, data) → ≤256  
3. →
```

音频数据格式

协控支持两种格式，自动识别：

WAV格式（带RIFF头）：

```
RIFF....WAVEfmt .....data....[PCM...]
```

- 8位PCM
- 单声道或双声道
- 采样率从WAV头读取（通常22050Hz）

原始PCM格式（无头）：

```
[PCM...]
```

- 8位无符号（静音值=128）
- 默认单声道 22050Hz

手动播放示例

```
import protocol as p  
import time  
  
# WAV  
with open('alert.dat', 'rb') as f:  
    wav_data = f.read()  
  
#  
p.uart_write(20, bytes([0x04]))  
time.sleep_ms(10)  
  
# 256  
chunk_size = 256  
for i in range(0, len(wav_data), chunk_size):  
    chunk = wav_data[i:i+chunk_size]  
    p.uart_write(22, bytes(chunk))  
    time.sleep_ms(5) #
```

5.7 录音控制指令（码16）

录音流程

```

1. uart_write(16, b'\x02') →
2. uart_write(16, b'\x02') → /
3. uart_write(16, b'\x00') →
4. PCM61440

```

子命令

子命令

十六进制

功能

2	0x02	开始录音 / 查询录音状态
0	0x00	停止录音

手动控制示例

```

import protocol as p
import time

#
p.uart_write(16, bytes([0x02]))
print("...")
time.sleep(2) # 2

#
p.uart_write(16, bytes([0x00]))
print("")

```

注意：手动控制只能启动/停止，录音数据的读取和保存由 `rec()` 函数内部完成，手动调用无法获取录音数据。

5.8 协议逆向技巧

Hook串口抓包

```

import protocol as p
import ubinascii

orig_write = p.uart_write

def hook_write(*args, **kwargs):
    cmd = args[0]
    data = args[1] if len(args) > 1 else b''
    hex_str = ubinascii.hexlify(bytes(data)).decode().upper()
    print("[TX] cmd=%d data=%s" % (cmd, hex_str))
    return orig_write(*args, **kwargs)

p.uart_write = hook_write

#
p.led_fill()
p.play('alert.dat')

```

常见问题

现象		原因	解决
点阵无反应	指令码或数据格式错误	确认指令码=8，数据5字节，高5位有效	
音频无声	未发送启动指令(20)	先发送 `uart_write(20, b"\x04")`	
音频卡顿	发送过快缓冲区溢出	每包之间加 `time.sleep_ms(5)`	
录音无文件	手动调用无法保存	使用 `rec('filename')` 函数	

第6章 5×5 LED 点阵编程指南

6.1 点阵硬件结构

5×5 LED 点阵由 HT 协控 MCU 直接驱动，采用行列扫描方式。ESP32 通过协控指令（码8）发送显示数据，无需直接操作GPIO。

```
 1  2  3  4  5
|  |  |  |  |
•  •  •  •  •  ← 1
•  •  •  •  •  ← 2
•  •  •  •  •  ← 3
•  •  •  •  •  ← 4
•  •  •  •  •  ← 5
```

6.2 数据格式

发送5字节数据，每字节对应一列，每字节高5位对应该列5个LED：

```
01: [bit7 bit6 bit5 bit4 bit3 bit2 bit1 bit0]
      |   |   |   |   |
      1   2   3   4   5
      ( )           ( )
```

6.3 基础操作

全屏点亮/熄灭

```
import protocol as p

p.led_fill() #
p.uart_write(8, bytes([0,0,0,0,0])) #
```

单列控制

```
# 1
p.uart_write(8, bytes([0xF8, 0, 0, 0, 0]))

# 3
p.uart_write(8, bytes([0, 0, 0xF8, 0, 0]))

# 5
p.uart_write(8, bytes([0, 0, 0, 0, 0xF8]))
```

单行控制

```
# 1
p.uart_write(8, bytes([0x80, 0x80, 0x80, 0x80, 0x80]))

# 3
p.uart_write(8, bytes([0x20, 0x20, 0x20, 0x20, 0x20]))

# 5
p.uart_write(8, bytes([0x08, 0x08, 0x08, 0x08, 0x08]))
```

单点控制

```
def set_pixel(x, y, on=True):
    """x=0-4(), y=0-4(0=)"""
    cols = [0, 0, 0, 0, 0]
    if on:
        cols[x] |= (1 << (7 - y)) # y=0→bit7, y=4→bit3
    p.uart_write(8, bytes(cols))

set_pixel(2, 2, True) #
```

6.4 Matrix 控制类

使用面向对象的方式控制点阵，更方便复杂图案操作：

```
import protocol as p
import time

class Matrix:
    def __init__(self):
        self.buffer = [0] * 5 # 5

    def clear(self):
        """
        self.buffer = [0] * 5

    def fill(self):
        """
        self.buffer = [0xF8] * 5

    def set_pixel(self, x, y, val=1):
        """ x=0-4, y=0-4(0=)"""
        if 0 <= x <= 4 and 0 <= y <= 4:
            if val:
                self.buffer[x] |= (1 << (7 - y))
            else:
```

```

        self.buffer[x] &= ~(1 << (7 - y))

def get_pixel(self, x, y):
    """
    if 0 <= x <= 4 and 0 <= y <= 4:
        return 1 if (self.buffer[x] >> (7 - y)) & 1 else 0
    return 0

def draw(self, pattern):
    """5X"""
    self.clear()
    for y in range(5):
        for x in range(5):
            if pattern[y][x] in ('X', 'x', '1', '#'):
                self.set_pixel(x, y, 1)

def show(self):
    """
    p.uart_write(8, bytes(self.buffer))

def shift_in_from_right(self, col_data):
    """
    self.buffer.pop(0)
    self.buffer.append(col_data & 0xF8)

#
m = Matrix()
m.draw([
    "XXXXX",
    "X..X",
    "X.X.X",
    "X..X",
    "XXXXX",
])
m.show()
time.sleep(1)
m.clear()
m.show()

```

6.5 图案设计方法

字符串图案格式

用5行字符串表示图案，每行5个字符，**x**表示亮，**.**表示灭：

```

#
heart = [
    ".X.X.", # 1
    "XXXXX", # 2
    "XXXXX", # 3
    ".XXX.", # 4
    "..X..", # 5
]

#
smile = [
    ".XXX.",
    "X.X.X",

```

```

        "XXXXX",
        "X..X..",
        ".XXX.",
    ]

#
arrow_up = [
    "..X..",
    ".XXX.",
    "XXXXX",
    "..X..",
    "..X..",
]

```

常用图案库

```

PATTERNS = {
    'heart': [".X.X.", "XXXXX", "XXXXX", ".XXX.", "..X.."],
    'smile': [".XXX.", "X.X.X", "XXXXX", "X.X.X", ".XXX."],
    'frown': [".XXX.", "X.X.X", "XXXXX", "X.X.X", "X...X"],
    'arrow_up': [".X..", ".XXX.", "XXXXX", "..X..", "..X.."],
    'arrow_down': [".X..", "..X..", "XXXXX", ".XXX.", "..X.."],
    'square': ["XXXXX", "X...X", "X...X", "X...X", "XXXXX"],
    'circle': [".XXX.", "X...X", "X...X", "X...X", ".XXX."],
    'star': [".X..", ".XXX.", "XXXXX", ".X.X.", "X...X"],
    'cross': [".X..", "..X..", "XXXXX", "..X..", ".X.."],
    'x': ["X...X", ".X.X.", ".X..", ".X.X.", "X...X"],
    'check': [".....", "...X", "...X", "X.X..", ".X..."],
    '0': [".XXX.", "X...X", "X...X", "X...X", ".XXX."],
    '1': [".X..", ".XX..", "..X..", ".X..", ".XXX."],
    '2': [".XXX.", "X...X", ".X..", ".X..", "XXXXX"],
    '3': ["XXXXX", ".X..", ".XXX.", ".X..", "XXXXX"],
    '4': ["X...X", "X...X", "XXXXX", "...X", "...X"],
    '5': ["XXXXX", "X...", "XXXX.", "...X", "XXXX."],
    '6': [".XXX.", "X...", "XXXX.", "X...X", ".XXX."],
    '7': ["XXXXX", "...X", "...X.", ".X..", ".X..."],
    '8': [".XXX.", "X...X", ".XXX.", "X...X", ".XXX."],
    '9': [".XXX.", "X...X", ".XXXX", "...X", ".XXX."],
}

```

6.6 滚动文字

```

def scroll_text(m, text, patterns, delay=0.12):
    """
    m: Matrix
    text:
    patterns:
    delay: ()
    """
    import time
    #
    cols = []
    for ch in text:
        if ch in patterns:
            pattern = patterns[ch]
            #

```

```

        for x in range(5):
            col = 0
            for y in range(5):
                if pattern[y][x] in ('X', 'x', '1'):
                    col |= (1 << (7 - y))
            cols.append(col)
        cols.append(0) #
    else:
        cols.extend([0, 0, 0]) #

#
m.clear()
for col in cols:
    m.shift_in_from_right(col)
    m.show()
    time.sleep(delay)
#
for _ in range(5):
    m.shift_in_from_right(0)
    m.show()
    time.sleep(delay)

#
m = Matrix()
scroll_text(m, "2026", PATTERNS, delay=0.15)

```

6.7 动画效果

心跳动画

```

def heartbeat(m, times=3):
    big = [".X.X.", "XXXXX", "XXXXX", ".XXX.", "..X.."]
    small = [".....", ".X.X.", ".XXX.", "..X..", "....."]
    for _ in range(times):
        m.draw(big); m.show(); time.sleep(0.15)
        m.draw(small); m.show(); time.sleep(0.15)
        m.draw(big); m.show(); time.sleep(0.3)

```

旋转加载动画

```

def loading(m, rounds=2):
    frames = [
        [".X..", "...X.", ".....", ".....", "....."],
        ["....X.", "....X", ".....", ".....", "....."],
        ["....X", ".....", "X....", ".....", "....."],
        ["......", ".....", ".X...", "X....", "....."],
        ["......", ".....", "..X..", ".X...", "....."],
        ["......", ".....", "...X.", "...X.", "....."],
        ["......", "....X", "....X", "...X.", "....."],
        [".X..", ".X...", "X....", ".....", "....."],
    ]
    for _ in range(rounds):
        for frame in frames:
            m.draw(frame); m.show(); time.sleep(0.08)

```

闪烁效果

```
def blink(m, pattern, times=3, delay=0.15):
    for _ in range(times):
        m.draw(pattern); m.show(); time.sleep(delay)
        m.clear(); m.show(); time.sleep(delay)
```

6.8 游戏开发基础

贪吃蛇核心逻辑

```
class SnakeGame:
    def __init__(self, matrix):
        self.m = matrix
        self.snake = [(2, 2)] #
        self.direction = (1, 0) #
        self.food = self.new_food()
        self.score = 0

    def new_food(self):
        import urandom
        while True:
            f = (urandom.randint(0,4), urandom.randint(0,4))
            if f not in self.snake:
                return f

    def update(self):
        #
        head = self.snake[0]
        new_head = ((head[0] + self.direction[0]) % 5,
                    (head[1] + self.direction[1]) % 5)
        #
        if new_head in self.snake:
            return False
        self.snake.insert(0, new_head)
        #
        if new_head == self.food:
            self.score += 1
            self.food = self.new_food()
        else:
            self.snake.pop()
        return True

    def draw(self):
        self.m.clear()
        for x, y in self.snake:
            self.m.set_pixel(x, y, 1)
        fx, fy = self.food
        self.m.set_pixel(fx, fy, 1)
        self.m.show()

    def set_direction(self, dx, dy):
        #
        if (dx, dy) != (-self.direction[0], -self.direction[1]):
            self.direction = (dx, dy)
```

第7章 音频编程指南

7.1 音频系统架构



7.2 播放音频

播放内置音效

```
import protocol as p

p.play('alert.dat') #
p.play('right.dat') #
p.play('wrong.dat') #
p.play('funny.dat') #
p.play('tech.dat') #
```

播放自定义音频

将WAV文件上传到设备后播放：

```
import protocol as p
p.play('my_sound.wav')
```

播放录音

```
import protocol as p

#
p.rec('my_voice')

#
p.play('my_voice')
```

7.3 音频格式详解

WAV文件结构

```

0      4      "RIFF"
4      4      -8
8      4      "WAVE"
12     4      "fmt "
16     4      fmt(16)
20     2      (1=PCM)
22     2      (1=,2=)
24     4      (22050)
28     4      (xx/8)
32     2      (x/8)
34     2      (8)
36     4      "data"
40     4
44     N      PCM

```

8位PCM数据格式

- 无符号8位整数
- 静音值 = 128 (0x80)
- 最大值 = 255 (0xFF)
- 最小值 = 0 (0x00)
- 波形围绕128上下波动

```

# 122050Hz440Hz A
import math
import struct

sample_rate = 22050
freq = 440
duration = 1.0
num_samples = int(sample_rate * duration)

pcm_data = bytearray()
for i in range(num_samples):
    t = i / sample_rate
    sample = 128 + int(60 * math.sin(2 * math.pi * freq * t))
    pcm_data.append(max(0, min(255, sample)))

# PCM
with open('tone.pcm', 'wb') as f:
    f.write(pcm_data)

```

7.4 自定义音效制作

方法一：电脑制作WAV后上传

1. 使用 Audacity 等音频编辑软件 2. 设置：8位、22050Hz、单声道 3. 导出为WAV格式 4. 通过 Thonny 上传到设备 5. 调用 `p.play('filename.wav')`

方法二：MicroPython代码生成

```
import protocol as p
import math

def generate_tone(freq, duration, volume=60, sample_rate=22050):
    """
    num_samples = int(sample_rate * duration)
    pcm = bytearray()
    for i in range(num_samples):
        t = i / sample_rate
        #
        fade = min(1.0, i / 500, (num_samples - i) / 500)
        sample = 128 + int(volume * fade * math.sin(2 * math.pi * freq * t))
        pcm.append(max(0, min(255, sample)))
    return bytes(pcm)

#
tone = generate_tone(440, 0.5) # 440Hz, 0.5

# PCM
p.uart_write(20, bytes([0x04]))
import time
time.sleep_ms(10)
chunk_size = 256
for i in range(0, len(tone), chunk_size):
    p.uart_write(22, tone[i:i+chunk_size])
    time.sleep_ms(5)
```

方法三：音效合成（蜂鸣/警报/爆炸）

```
def generate_beep(freq, duration, volume=80):
    """
    return generate_tone(freq, duration, volume)

def generate_alarm(duration=1.0):
    """
    sample_rate = 22050
    num_samples = int(sample_rate * duration)
    pcm = bytearray()
    for i in range(num_samples):
        t = i / sample_rate
        # 400-800Hz
        freq = 400 + 400 * abs(math.sin(2 * math.pi * 2 * t))
        sample = 128 + int(70 * math.sin(2 * math.pi * freq * t))
        pcm.append(max(0, min(255, sample)))
    return bytes(pcm)

def generate_explosion(duration=0.5):
    """+"""
    import urandom
    sample_rate = 22050
    num_samples = int(sample_rate * duration)
    pcm = bytearray()
    for i in range(num_samples):
        #
        decay = 1.0 - (i / num_samples)
        noise = urandom.randint(0, 255)
        sample = int(128 + (noise - 128) * decay)
```

```
pcm.append(max(0, min(255, sample)))
return bytes(pcm)
```

7.5 录音功能

基础录音

```
import protocol as p

p.rec('myrecording') # 2.78 myrecording
p.play('myrecording') #
```

录音参数

参数	值
格式	原始8位PCM (无WAV头)
采样率	22050 Hz
位深	8位
声道	单声道
固定大小	61440 字节
时长	约2.78秒
提示音	开始播放 record_start.dat, 结束播放 record_end.dat

多段录音拼接

```
import protocol as p
import time

def record_multiple(segments, delay=0.3):
    """
    all_data = b''
    for i in range(segments):
        fname = 'seg_%d' % i
        print(" %d/%d ..." % (i+1, segments))
        p.rec(fname)
        with open(fname, 'rb') as f:
            all_data += f.read()
        time.sleep(delay)
    #
    import os
    os.remove(fname)
    #
    with open('combined', 'wb') as f:
        f.write(all_data)
    print(":", len(all_data), "")
    return 'combined'

result = record_multiple(3)
p.play(result)
```

录音转WAV导出

```

import struct

def pcm_to_wav(pcm_file, wav_file, sample_rate=22050):
    """PCMWAV"""
    with open(pcm_file, 'rb') as f:
        pcm_data = f.read()

    num_channels = 1
    bits_per_sample = 8
    byte_rate = sample_rate * num_channels * bits_per_sample // 8
    block_align = num_channels * bits_per_sample // 8
    data_size = len(pcm_data)
    file_size = 36 + data_size

    header = struct.pack('<4sI4s4sIHIIHH4sI',
        b'RIFF', file_size, b'WAVE',
        b'fmt ', 16, 1, num_channels,
        sample_rate, byte_rate, block_align, bits_per_sample,
        b'data', data_size)

    with open(wav_file, 'wb') as f:
        f.write(header)
        f.write(pcm_data)

    print(" %s (%d )" % (wav_file, 44 + data_size))

#
pcm_to_wav('myrecording', 'myrecording.wav')

```

7.6 音乐律动可视化

音量条可视化

```

import protocol as p
import time

def play_with_visualizer(filename, chunk_size=256):
    """
    #
    with open(filename, 'rb') as f:
        data = f.read()

    # WAV
    if data[:4] == b'RIFF':
        data_offset = 44 # WAV
        data = data[data_offset:]

    #
    p.uart_write(20, bytes([0x04]))
    time.sleep_ms(10)

    #
    for i in range(0, len(data), chunk_size):
        chunk = data[i:i+chunk_size]
        if len(chunk) < chunk_size:
            chunk = chunk + b'\x80' * (chunk_size - len(chunk))

```

```

#
p.uart_write(22, bytes(chunk))

# 8PCM128
vol = sum(abs(b - 128) for b in chunk) // len(chunk)
level = int(vol * 6 / 128) # 0-5

#
cols = [0, 0, 0, 0, 0]
for col in range(5):
    col_level = level
    if col == 2: #
        col_level = min(5, level + 1)
    elif col in (0, 4): #
        col_level = max(0, level - 1)
    for row in range(5 - col_level, 5):
        cols[col] |= (1 << (7 - row))
p.uart_write(8, bytes(cols))

time.sleep_ms(5)

#
for i in range(5, -1, -1):
    cols = [0, 0, 0, 0, 0]
    for col in range(5):
        for row in range(5 - i, 5):
            cols[col] |= (1 << (7 - row))
    p.uart_write(8, bytes(cols))
    time.sleep_ms(30)
p.uart_write(8, bytes([0,0,0,0,0]))

#
play_with_visualizer('electric.dat')

```

频谱风格可视化

```

def play_spectrum(filename, chunk_size=128):
    """5"""
    with open(filename, 'rb') as f:
        data = f.read()
    if data[:4] == b'RIFF':
        data = data[44:]

    p.uart_write(20, bytes([0x04]))
    time.sleep_ms(10)

    prev_levels = [0, 0, 0, 0, 0]

    for i in range(0, len(data), chunk_size):
        chunk = data[i:i+chunk_size]
        if len(chunk) < chunk_size:
            chunk = chunk + b'\x80' * (chunk_size - len(chunk))

        p.uart_write(22, bytes(chunk))

    # chunk5
    seg_size = len(chunk) // 5

```

```
levels = []
for s in range(5):
    seg = chunk[s*seg_size:(s+1)*seg_size]
    vol = sum(abs(b - 128) for b in seg) // len(seg)
    level = int(vol * 6 / 128)
    #
    level = (prev_levels[s] * 2 + level) // 3
    levels.append(level)
prev_levels = levels

#
cols = [0, 0, 0, 0, 0]
for col in range(5):
    for row in range(5 - levels[col], 5):
        cols[col] |= (1 << (7 - row))
p.uart_write(8, bytes(cols))

time.sleep_ms(3)

p.uart_write(8, bytes([0,0,0,0,0]))
```

7.7 音频常见问题

问题

原因

解决

播放无声	文件格式不支持	确认8位22050Hz WAV或原始PCM
声音卡顿	发送过快	每包之间加`time.sleep_ms(5)`
声音过小	PCM数据幅值不够	放大幅值（但不要超过0-255范围）
爆音/杂音	波形突变	加入淡入淡出（首尾500采样渐变）
play()无反应	文件名错误	确认文件存在于根目录
录音无声	麦克风问题	检查麦克风连接，或硬件故障
录音播放太快	采样率不匹配	原始PCM默认22050Hz，确认播放设置

第8章 按键与输入指南

8.1 按键硬件

按键

GPIO

输入模式

触发电平

A	GPIO10	下拉输入 (PULL_DOWN)	按下=高电平(1)
B	GPIO1	下拉输入 (PULL_DOWN)	按下=高电平(1)

C	GPIO3	下拉输入 (PULL_DOWN)	按下=高电平(1)
---	-------	------------------	-----------

重要： 按键采用下拉输入，按下时GPIO为高电平，松开时为低电平。与常见的上拉输入（按下低电平）相反。

8.2 基础按键读取

```
from machine import Pin
import time

#
key_a = Pin(10, Pin.IN, Pin.PULL_DOWN)
key_b = Pin(1, Pin.IN, Pin.PULL_DOWN)
key_c = Pin(3, Pin.IN, Pin.PULL_DOWN)

#
while True:
    if key_a.value() == 1:
        print("A")
    if key_b.value() == 1:
        print("B")
    if key_c.value() == 1:
        print("C")
    time.sleep_ms(50)
```

8.3 按键消抖

机械按键按下时会产生抖动（约10-20ms），需要消抖处理：

方法一：延时消抖

```
def read_key_debounced(pin, debounce_ms=20):
    if pin.value() == 1:
        time.sleep_ms(debounce_ms)
        if pin.value() == 1:
            while pin.value() == 1: #
                time.sleep_ms(10)
            return True
    return False

#
while True:
    if read_key_debounced(key_a):
        print("A")
    time.sleep_ms(10)
```

方法二：时间戳消抖（推荐）

```
last_press_time = 0
DEBOUNCE_MS = 200

def read_key():
    global last_press_time
    now = time.ticks_ms()
    if time.ticks_diff(now, last_press_time) < DEBOUNCE_MS:
        return None
```

```
    if key_a.value() == 1:
        last_press_time = now
        return 'A'
    if key_b.value() == 1:
        last_press_time = now
        return 'B'
    if key_c.value() == 1:
        last_press_time = now
        return 'C'
    return None

#
while True:
    key = read_key()
    if key == 'A':
        print("A")
    elif key == 'B':
        print("B")
    elif key == 'C':
        print("C")
    time.sleep_ms(20)
```

8.4 按键状态机

处理按下、长按、松开等状态：

```
class Button:
    def __init__(self, pin, name):
        self.pin = pin
        self.name = name
        self.last_state = 0
        self.press_time = 0
        self.long_press_ms = 1000 #

    def update(self):
        """ : None, 'press', 'release', 'long_press' """
        current = self.pin.value()
        now = time.ticks_ms()
        event = None

        if current == 1 and self.last_state == 0:
            #
            self.press_time = now
            event = 'press'
        elif current == 0 and self.last_state == 1:
            #
            duration = time.ticks_diff(now, self.press_time)
            if duration >= self.long_press_ms:
                event = 'long_press'
            else:
                event = 'release'
        elif current == 1 and self.last_state == 1:
            #
            duration = time.ticks_diff(now, self.press_time)
            if duration >= self.long_press_ms and not hasattr(self, '_long_fired'):
                self._long_fired = True
                event = 'long_press'
```

```
        if current == 0:
            self._long_fired = False

        self.last_state = current
        return event

#
btn_a = Button(key_a, 'A')
btn_b = Button(key_b, 'B')
btn_c = Button(key_c, 'C')

while True:
    for btn in [btn_a, btn_b, btn_c]:
        event = btn.update()
        if event == 'press':
            print(btn.name, "")
        elif event == 'release':
            print(btn.name, "")
        elif event == 'long_press':
            print(btn.name, "")
    time.sleep_ms(10)
```

8.5 组合按键检测

```
def check_combination():
    """
    a = key_a.value()
    b = key_b.value()
    c = key_c.value()

    if a and b:
        return 'A+B'
    elif a and c:
        return 'A+C'
    elif b and c:
        return 'B+C'
    elif a and b and c:
        return 'A+B+C'
    elif a:
        return 'A'
    elif b:
        return 'B'
    elif c:
        return 'C'
    return None

#
last_combo = 0
while True:
    now = time.ticks_ms()
    if time.ticks_diff(now, last_combo) > 300:
        combo = check_combination()
        if combo:
            print(":", combo)
            last_combo = now
    time.sleep_ms(20)
```

8.6 使用 hal.key_map

`protocol.hal.key_map` 提供了预定义的按键Pin对象：

```
import protocol as p

#
key_a = p.hal.key_map['A']
key_b = p.hal.key_map['B']
key_c = p.hal.key_map['C']

# hal.key_map Pin
#
key_a.init(Pin.IN, Pin.PULL_DOWN)
key_b.init(Pin.IN, Pin.PULL_DOWN)
key_c.init(Pin.IN, Pin.PULL_DOWN)

print("A:", key_a.value())
```

8.7 按键应用示例

菜单导航

```
menu_items = ["1", "2", "3", "4"]
menu_index = 0

def handle_menu():
    global menu_index
    key = read_key()
    if key == 'A':
        menu_index = (menu_index + 1) % len(menu_items)
        print(":", menu_items[menu_index])
    elif key == 'B':
        menu_index = (menu_index - 1) % len(menu_items)
        print(":", menu_items[menu_index])
    elif key == 'C':
        print(":", menu_items[menu_index])
        return True #
    return False
```

计数器

```
count = 0
while True:
    key = read_key()
    if key == 'A':
        count += 1
        print(":", count)
    elif key == 'B':
        count -= 1
        print(":", count)
    elif key == 'C':
        count = 0
        print("")
    time.sleep_ms(20)
```

第9章 加速度计使用指南

9.1 硬件规格

参数	规格
型号	SC7A20
类型	三轴MEMS加速度计
接口	SoftI2C
地址	0x18 (7位地址)
SDA	GPIO6
SCL	GPIO7
时钟频率	500kHz
量程	±2g (默认)
分辨率	12位
灵敏度	1mg/LSB (±2g)
输出数据率	100Hz (可配置)

9.2 基础读取

```
import protocol as p

i2c = p.hal.i2c_map['ACC']
ACC_ADDR = 0x18

#
i2c.writeto_mem(ACC_ADDR, 0x20, bytes([0x57])) # 100Hz, , XYZ
i2c.writeto_mem(ACC_ADDR, 0x23, bytes([0x00])) # ±2g

def read_accel():
    """ (x, y, z) """
    data = i2c.readfrom_mem(ACC_ADDR, 0x28, 6)
    x = (data[1] << 8) | data[0]
    y = (data[3] << 8) | data[2]
    z = (data[5] << 8) | data[4]
    # 16
    if x >= 32768: x -= 65536
    if y >= 32768: y -= 65536
    if z >= 32768: z -= 65536
    return (x, y, z)

#
x, y, z = read_accel()
print("X:", x, "Y:", y, "Z:", z)
```

9.3 寄存器说明

寄存器

	地址	说明
WHO_AM_I	0x0F	设备ID (读取验证)
CTRL_REG1	0x20	输出数据率、低功耗模式、轴使能
CTRL_REG2	0x21	高通滤波器设置
CTRL_REG3	0x22	中断配置
CTRL_REG4	0x23	量程、自测试、SPI模式
CTRL_REG5	0x24	中断使能、FIFO
OUT_X_L	0x28	X轴低字节
OUT_X_H	0x29	X轴高字节
OUT_Y_L	0x2A	Y轴低字节
OUT_Y_H	0x2B	Y轴高字节
OUT_Z_L	0x2C	Z轴低字节
OUT_Z_H	0x2D	Z轴高字节
INT1_CFG	0x30	中断1配置
INT1_SRC	0x31	中断1源
INT1_THS	0x32	中断1阈值
INT1_DUR	0x33	中断1持续时间

CTRL_REG1 (0x20) 位定义

```

bit7: ODR3  1
bit6: ODR2  |
bit5: ODR1  | 0000=, 0001=1Hz, 0010=10Hz, 0011=25Hz
bit4: ODR0  | 0100=50Hz, 0101=100Hz, 0110=200Hz, 0111=400Hz
bit3: LPen   (1=)
bit2: Zen    Z (1=)
bit1: Yen    Y (1=)
bit0: Xen    X (1=)

```

常用配置：

- **0x57** = 0101 0111 → 100Hz, 正常模式, XYZ使能
- **0x47** = 0100 0111 → 50Hz, 正常模式, XYZ使能
- **0x67** = 0110 0111 → 200Hz, 正常模式, XYZ使能

CTRL_REG4 (0x23) 位定义

```

bit7: BDU      (1=MSBLSB)
bit6: BLE      / (0=, 1=)
bit5: FS1      1
bit4: FS0      | 00=±2g, 01=±4g, 10=±8g, 11=±16g
bit3: HR       (1=)
bit2: 0
bit1: ST1      1
bit0: ST0      | 00=, 01=0, 10=1

```

常用配置：

- **0x00** = ±2g, 正常模式
- **0x10** = ±4g

- $0x20 = \pm 8g$

9.4 单位转换

原始值转换为重力加速度(g):

```
def raw_to_g(raw, range_g=2):
    """g
    ±2g: 1mg/LSB → /1000 = g
    ±4g: 2mg/LSB → /500 = g
    ±8g: 4mg/LSB → /250 = g
    """
    sensitivity = {2: 1000, 4: 500, 8: 250, 16: 125}
    return raw / sensitivity.get(range_g, 1000)

x, y, z = read_accel()
print("X: %.3fg Y: %.3fg Z: %.3fg" % (
    raw_to_g(x), raw_to_g(y), raw_to_g(z)))
```

9.5 姿态检测

倾斜角度计算

```
import math

def get_angles():
    """
    x, y, z = read_accel()
    xg = raw_to_g(x)
    yg = raw_to_g(y)
    zg = raw_to_g(z)

    #
    pitch = math.atan2(xg, math.sqrt(yg*yg + zg*zg)) * 180 / math.pi
    roll = math.atan2(yg, math.sqrt(xg*xg + zg*zg)) * 180 / math.pi

    return pitch, roll

#
pitch, roll = get_angles()
print(": %.1f° : %.1f°" % (pitch, roll))
```

方向判断

```
def get_orientation():
    """
    x, y, z = read_accel()
    xg = raw_to_g(x)
    yg = raw_to_g(y)
    zg = raw_to_g(z)

    threshold = 0.5

    if abs(zg) > threshold and abs(xg) < threshold and abs(yg) < threshold:
        return ' ' if zg > 0 else ' '
    elif abs(xg) > threshold and abs(yg) < threshold and abs(zg) < threshold:
        return ' ' if xg > 0 else ' '
    elif abs(yg) > threshold and abs(xg) < threshold and abs(zg) < threshold:
        return ' ' if yg > 0 else ' '
    else:
        return ' ' if zg > 0 else ' '
    """
```

```
elif abs(yg) > threshold and abs(xg) < threshold and abs(zg) < threshold:
    return '1' if yg > 0 else '1'
else:
    return ''
```

9.6 摇晃检测

```
class ShakeDetector:
    def __init__(self, threshold=70000, cooldown=60):
        self.threshold = threshold
        self.cooldown = cooldown
        self.cooldown_counter = 0
        self.last_x = 0
        self.last_y = 0
        self.last_z = 0
        #
        self.last_x, self.last_y, self.last_z = read_accel()

    def update(self):
        """ True """
        if self.cooldown_counter > 0:
            self.cooldown_counter -= 1
            return False

        x, y, z = read_accel()
        dx = abs(x - self.last_x)
        dy = abs(y - self.last_y)
        dz = abs(z - self.last_z)
        total = dx + dy + dz

        self.last_x = x
        self.last_y = y
        self.last_z = z

        if total > self.threshold:
            self.cooldown_counter = self.cooldown
            return True
        return False

#
detector = ShakeDetector(threshold=70000)
while True:
    if detector.update():
        print("")
    time.sleep_ms(20)
```

阈值参考（基于实测数据）：

阈值		
灵敏度		触发力度
50000	高	轻微晃动
70000	中	明显摇晃（推荐）
90000	低	用力摇晃
120000	很低	剧烈摇晃

9.7 计步器

```
class Pedometer:
    def __init__(self, threshold=15000, debounce=10):
        self.threshold = threshold
        self.debounce = debounce
        self.debounce_counter = 0
        self.steps = 0
        self.last_z = 0
        self.peak = False

    def update(self):
        """
        if self.debounce_counter > 0:
            self.debounce_counter -= 1
            return self.steps

        x, y, z = read_accel()
        #
        total = math.sqrt(x*x + y*y + z*z)

        # Z
        if total > self.threshold and not self.peak:
            self.peak = True
            self.steps += 1
            self.debounce_counter = self.debounce
        elif total < self.threshold * 0.7 and self.peak:
            self.peak = False

        return self.steps

#
pedometer = Pedometer()
while True:
    steps = pedometer.update()
    print(":", steps)
    time.sleep_ms(50)
```

9.8 水平仪

```
def level_meter():
    """
    x, y, z = read_accel()
    xg = raw_to_g(x)
    yg = raw_to_g(y)

    # 5x5
    center_x = 2 + int(xg * 2) #
    center_y = 2 + int(yg * 2) #
    center_x = max(0, min(4, center_x))
    center_y = max(0, min(4, center_y))

    #
    cols = [0, 0, 0, 0, 0]
    cols[center_x] |= (1 << (7 - center_y))
    p.uart_write(8, bytes(cols))

#
while True:
    level_meter()
    time.sleep_ms(50)
```

9.9 自由落体检测

```
def detect_free_fall():
    """0"""
    x, y, z = read_accel()
    total = math.sqrt(x*x + y*y + z*z)
    # 0
    return total < 5000 # 0.5g

#
while True:
    if detect_free_fall():
        print("")
        p.play('alert.dat')
    time.sleep_ms(20)
```

9.10 加速度计常见问题

问题	原因	解决
读取值全为0	未初始化	先写 CTRL_REG1(0x20)=0x57
值不变化	轴未使能	确认 CTRL_REG1 的 bit0-2 为1
值跳动大	数据率太高	降低ODR到50Hz(0x47)
量程不够	超过±2g	设置 CTRL_REG4(0x23)=0x10 (±4g)
角度不准	未校准	平放时记录基准值并减去
I2C通信失败	地址错误	确认地址0x18, 扫描I2C总线

第10章 文件系统指南

10.1 文件系统结构

ESP32-C3 使用 LittleFS 文件系统，挂载在根目录 `/`。所有用户文件都在根目录下。

```
/
├─ boot.py          #
├─ main.py          # boot.py
├─ alert.dat        #
├─ ...              #
└─ lib/             #
```

10.2 基本文件操作

```
import os

#
print(os.listdir())
print(os.listdir('/'))

#
os.mkdir('mydir')

#
os.rmdir('mydir')

#
os.rename('old.txt', 'new.txt')

#
os.remove('test.txt')

#
stat = os.stat('main.py')
print(":", stat[6], "")
print(":", stat[7])

#
print(os.getcwd())

#
os.chdir('/mydir')
```

10.3 文件读写

文本文件

```
#
with open('data.txt', 'w') as f:
    f.write('Hello AlphaPi Zero\n')
    f.write('\n')

#
with open('data.txt', 'a') as f:
    f.write('\n')

#
with open('data.txt', 'r') as f:
    content = f.read()
    print(content)

#
with open('data.txt', 'r') as f:
    for line in f:
        print(line.strip())
```

二进制文件

```
#
data = bytes([0x00, 0x01, 0x02, 0xFF])
with open('binary.bin', 'wb') as f:
    f.write(data)

#
with open('binary.bin', 'rb') as f:
    data = f.read()
    print(data.hex())

#
with open('alert.dat', 'rb') as f:
    header = f.read(44) # WAV
    print("WAV:", header[:4])
```

按位置读写

```
with open('data.txt', 'r+') as f:
    f.seek(10) # 10
    print(f.read(5)) # 5
    f.seek(0) #
    f.write('X') # 1
```

10.4 JSON 数据存储

```
import ujson

#
config = {
    'name': 'MyDevice',
    'version': 1,
    'settings': {'brightness': 80, 'volume': 50}
}
with open('config.json', 'w') as f:
```

```
    ujson.dump(config, f)

#
with open('config.json', 'r') as f:
    config = ujson.load(f)
print(config['name'])
print(config['settings']['volume'])
```

10.5 CSV 数据记录

```
import time

# CSV
with open('accel_log.csv', 'w') as f:
    f.write('time,x,y,z\n') #

for i in range(100):
    x, y, z = read_accel()
    with open('accel_log.csv', 'a') as f:
        f.write('%d,%d,%d,%d\n' % (time.ticks_ms(), x, y, z))
    time.sleep_ms(100)
```

10.6 存储空间管理

```
import os

#
def get_dir_size(path='/'):
    total = 0
    for f in os.listdir(path):
        try:
            stat = os.stat(path + '/' + f)
            total += stat[6]
        except:
            pass
    return total

print(":", get_dir_size(), "")

# Flashesp
import esp
print("Flash:", esp.flash_size(), "")
print("Flash:", esp.flash_user_start(), "")
```

10.7 内置文件说明

文件		说明
	大小	
boot.py	90B	启动引导，检查协控状态，条件触发工厂
main.py	55B	主程序入口，初始化文件和硬件自检
logo2.bmp	8694B	开机Logo，24×120像素24位BMP（5帧
*.dat	16-45KB	13个内置音效文件，WAV格式

10.8 文件上传下载

通过 Thonny 上传

1. 在 Thonny 中打开本地文件 2. 右键文件 → "上传到 /" 3. 或使用 "文件" 面板拖拽上传

通过 ampy 命令行

```
#
ampy --port COM3 put main.py

#
ampy --port COM3 get main.py

#
ampy --port COM3 ls

#
ampy --port COM3 rm test.txt
```

通过 REPL 手动写入

```
#
content = '''import protocol as p
p.led_fill()
'''
with open('test.py', 'w') as f:
    f.write(content)
```

10.9 数据备份与恢复

```
import os

#
backup = {}
for fname in os.listdir():
    try:
        with open(fname, 'rb') as f:
            backup[fname] = f.read()
    except:
        pass

#
import ujson
# JSONbase64
import ubinascii
```

```
backup_b64 = {k: ubinascii.b2a_base64(v).decode() for k, v in backup.items()}
with open('backup.json', 'w') as f:
    ujson.dump(backup_b64, f)

#
with open('backup.json', 'r') as f:
    backup_b64 = ujson.load(f)
for fname, b64data in backup_b64.items():
    data = ubinascii.a2b_base64(b64data)
    with open(fname, 'wb') as f:
        f.write(data)
```

第11章 完整示例代码集

11.1 秒表

```
import protocol as p
from machine import Pin
import time

#
key_a = Pin(10, Pin.IN, Pin.PULL_DOWN)
key_b = Pin(1, Pin.IN, Pin.PULL_DOWN)
key_c = Pin(3, Pin.IN, Pin.PULL_DOWN)

#
def show_raw(cols):
    p.uart_write(8, bytes([c << 3 for c in cols]))

def clear_display():
    show_raw([0,0,0,0,0])

#
DIGITS = {
    '0': [".XXX.", "X...X", "X...X", "X...X", ".XXX."],
    '1': [".X..", ".XX..", "..X..", "..X..", ".XXX."],
    '2': [".XXX.", "X...X", "..X..", ".X...", "XXXXX"],
    '3': [".XXXXX", ".X..", ".XXX.", ".X..", "XXXXX"],
    '4': [".X...X", "X...X", "XXXXX", "...X", "...X"],
    '5': [".XXXXX", "X...", "XXXX.", "...X", "XXXX."],
    '6': [".XXX.", "X...", "XXXX.", "X...X", ".XXX."],
    '7': [".XXXXX", "...X", ".X..", ".X..", ".X..."],
    '8': [".XXX.", "X...X", ".XXX.", "X...X", ".XXX."],
    '9': [".XXX.", "X...X", ".XXXX", "...X", ".XXX."],
    'P': [".XXX.", "X..X.", "XXXX.", "X...", "X..."]
}

def show_char(ch):
    if ch not in DIGITS:
        clear_display()
        return
    pattern = DIGITS[ch]
    cols = [0,0,0,0,0]
```

```
    for y in range(5):
        for x in range(5):
            if pattern[y][x] == 'X':
                cols[x] |= (1 << (7 - y))
    show_raw(cols)

#
last_press = 0
def read_key():
    global last_press
    now = time.ticks_ms()
    if time.ticks_diff(now, last_press) < 250:
        return None
    if key_a.value() == 1: last_press = now; return 'A'
    if key_b.value() == 1: last_press = now; return 'B'
    if key_c.value() == 1: last_press = now; return 'C'
    return None

#
state = 0 # 0=, 1=, 2=
start_ms = 0
elapsed = 0
last_sec = -1

clear_display()
print(": A=/ B= C=")

while True:
    key = read_key()

    if state == 0 and key == 'A':
        state = 1
        start_ms = time.ticks_ms()
        last_sec = -1
        print("")
    elif state == 1 and key == 'B':
        state = 2
        elapsed = time.ticks_diff(time.ticks_ms(), start_ms) / 1000
        show_char('P')
        print(": %.1fs" % elapsed)
    elif state == 1 and key == 'C':
        elapsed = time.ticks_diff(time.ticks_ms(), start_ms) / 1000
        state = 0
        print(": %.1fs" % elapsed)
        clear_display()
        last_sec = -1
    elif state == 2 and key == 'A':
        state = 1
        start_ms = time.ticks_ms() - int(elapsed * 1000)
        last_sec = -1
        print("")
    elif state == 2 and key == 'C':
        state = 0
        print(": %.1fs" % elapsed)
        clear_display()
        last_sec = -1

    if state == 1:
```

```

        current_sec = int(time.time() - start_ms) / 1000
    if current_sec != last_sec:
        last_sec = current_sec
        show_char(str(current_sec % 10))

    time.sleep_ms(20)

```

11.2 摇骰子

```

import protocol as p
from machine import Pin
import time
import urandom

key_a = Pin(10, Pin.IN, Pin.PULL_DOWN)
key_b = Pin(1, Pin.IN, Pin.PULL_DOWN)
key_c = Pin(3, Pin.IN, Pin.PULL_DOWN)

def show_raw(cols):
    p.uart_write(8, bytes([c << 3 for c in cols]))
def clear_display():
    show_raw([0,0,0,0,0])
def pattern_to_cols(pattern):
    cols = [0,0,0,0,0]
    for y in range(5):
        for x in range(5):
            if pattern[y][x] == 'X':
                cols[x] |= (1 << (7 - y))
    return cols
def show_pattern(pattern):
    show_raw(pattern_to_cols(pattern))

DICE = {
    1: ["XXXXX", "X...X", "X.X.X", "X...X", "XXXXX"],
    2: ["XXXXX", "XX..X", "X...X", "X..XX", "XXXXX"],
    3: ["XXXXX", "XX..X", "X.X.X", "X..XX", "XXXXX"],
    4: ["XXXXX", "XX.XX", "X...X", "XX.XX", "XXXXX"],
    5: ["XXXXX", "XX.XX", "X.X.X", "XX.XX", "XXXXX"],
    6: ["XXXXX", "XX.XX", "XX.XX", "XX.XX", "XXXXX"],
}

FRAME = ["XXXXX", "X...X", "X...X", "X...X", "XXXXX"]
ALLON = ["XXXXX", "XXXXX", "XXXXX", "XXXXX", "XXXXX"]
CROSS = ["X...X", ".X.X.", ".X.X.", ".X.X.", "X...X"]

def roll_animation():
    for i in range(8):
        show_pattern(FRAME if i%2==0 else DICE[urandom.randint(1,6)])
        time.sleep_ms(40)
    for _ in range(3):
        show_pattern(CROSS); time.sleep_ms(50)
        show_pattern(ALLON); time.sleep_ms(30)
    for d in [60,70,90,110,140,180,230,300]:
        show_pattern(DICE[urandom.randint(1,6)])
        time.sleep_ms(d)

def result_animation(num):

```

```
show_pattern(ALLON); time.sleep_ms(80)
show_pattern(FRAME); time.sleep_ms(80)
show_pattern(DICE[num]); time.sleep_ms(200)
for _ in range(3):
    clear_display(); time.sleep_ms(100)
    show_pattern(DICE[num]); time.sleep_ms(150)
for _ in range(2):
    show_pattern(FRAME); time.sleep_ms(100)
    show_pattern(DICE[num]); time.sleep_ms(200)

def roll_dice():
    roll_animation()
    result = urandom.randint(1,6)
    result_animation(result)
    time.sleep(2)
    return result

last_press = 0
def read_key():
    global last_press
    now = time.ticks_ms()
    if time.ticks_diff(now, last_press) < 300: return None
    if key_a.value() == 1: last_press = now; return 'A'
    if key_b.value() == 1: last_press = now; return 'B'
    if key_c.value() == 1: last_press = now; return 'C'
    return None

clear_display()
show_pattern(DICE[1])
rolling = False

while True:
    key = read_key()
    if not rolling:
        if key == 'A':
            rolling = True
            roll_dice()
            rolling = False
        elif key == 'B':
            rolling = True
            n = urandom.randint(1,6)
            for _ in range(3):
                show_pattern(ALLON); time.sleep_ms(50)
                show_pattern(DICE[n]); time.sleep_ms(80)
            time.sleep(2)
            rolling = False
        elif key == 'C':
            clear_display()
            while read_key() != 'A':
                time.sleep_ms(20)
    time.sleep_ms(20)
```

11.3 反应速度测试器

```
import protocol as p
from machine import Pin
import time
import urandom

key_a = Pin(10, Pin.IN, Pin.PULL_DOWN)

def show_raw(cols):
    p.uart_write(8, bytes([c << 3 for c in cols]))
def clear_display():
    show_raw([0,0,0,0,0])

#
RED = [0xF8,0xF8,0xF8,0xF8,0xF8]
#
GREEN = [0,0,0x20,0,0]

best_time = None

print("A")

while True:
    #
    for i in range(3, 0, -1):
        print(i)
        time.sleep(1)

    # 1-4
    delay = urandom.randint(1000, 4000)
    start = time.ticks_ms()

    #
    while time.ticks_diff(time.ticks_ms(), start) < delay:
        show_raw(RED) #
        time.sleep_ms(50)

    #
    show_raw(GREEN)
    p.play('beep' if False else 'alert.dat') #
    react_start = time.ticks_ms()

    #
    while key_a.value() == 0:
        time.sleep_ms(5)

    react_time = time.ticks_diff(time.ticks_ms(), react_start)
    clear_display()

    print(": %d ms" % react_time)
    if best_time is None or react_time < best_time:
        best_time = react_time
        print("")

    print(": %d ms" % best_time)
    print("A")
    while key_a.value() == 0:
        time.sleep_ms(50)
```

```
time.sleep(0.5)
```

11.4 魔法8球 (随机答案)

```
import protocol as p
from machine import Pin
import time
import urandom

key_a = Pin(10, Pin.IN, Pin.PULL_DOWN)

def show_raw(cols):
    p.uart_write(8, bytes([c << 3 for c in cols]))
def clear_display():
    show_raw([0,0,0,0,0])

# 1-8
ANSWERS = [
    "", "", "", "",
    "", "", "", ""
]

# 8
ANSWER_PATTERNS = {
    1: [".X..", ".X.X.", "X...X", ".X.X.", "..X.."], #
    2: ["XXXXX", ".....", "XXXXX", ".....", "XXXXX"], # =
    3: ["X...X", ".X.X.", ".X..", ".X.X.", "X...X"], # X=
    4: [".X..", ".X..", "XXXXX", ".X..", ".X.."], # =
    5: [".XXX.", "X...X", "X...X", "X...X", ".XXX."], # =
    6: ["X...X", ".X.X.", ".X..", ".X.X.", "X...X"], # X=
    7: ["XXXXX", "...X", "...X.", ".X..", ".X..."], # =
    8: ["X...X", ".X.X.", ".X..", ".X.X.", "X...X"], # X=
}

def pattern_to_cols(pattern):
    cols = [0,0,0,0,0]
    for y in range(5):
        for x in range(5):
            if pattern[y][x] == 'X':
                cols[x] |= (1 << (7 - y))
    return cols

print("8A")

while True:
    #
    while key_a.value() == 0:
        time.sleep_ms(50)

    #
    for i in range(20):
        idx = urandom.randint(1, 8)
        show_raw(pattern_to_cols(ANSWER_PATTERNS[idx]))
        time.sleep_ms(50)

    #
    answer_idx = urandom.randint(1, 8)
```

```

    show_raw(pattern_to_cols(ANSWER_PATTERNS[answer_idx]))
    p.play('alert.dat')
    print(":", ANSWERS[answer_idx - 1])

    # 3
    time.sleep(3)
    clear_display()
    print("A")

```

11.5 简易钢琴（按键播放音调）

```

import protocol as p
from machine import Pin
import time
import math

key_a = Pin(10, Pin.IN, Pin.PULL_DOWN)
key_b = Pin(1, Pin.IN, Pin.PULL_DOWN)
key_c = Pin(3, Pin.IN, Pin.PULL_DOWN)

def generate_tone(freq, duration=0.3, volume=60, sample_rate=22050):
    """
    num_samples = int(sample_rate * duration)
    pcm = bytearray()
    for i in range(num_samples):
        t = i / sample_rate
        fade = min(1.0, i / 500, (num_samples - i) / 500)
        sample = 128 + int(volume * fade * math.sin(2 * math.pi * freq * t))
        pcm.append(max(0, min(255, sample)))
    return bytes(pcm)

def play_tone(freq, duration=0.3):
    """
    tone = generate_tone(freq, duration)
    p.uart_write(20, bytes([0x04]))
    time.sleep_ms(10)
    chunk_size = 256
    for i in range(0, len(tone), chunk_size):
        p.uart_write(22, tone[i:i+chunk_size])
        time.sleep_ms(5)

#
NOTES = {
    'A': 523, # C5
    'B': 659, # E5
    'C': 784, # G5
}

last_press = 0
def read_key():
    global last_press
    now = time.ticks_ms()
    if time.ticks_diff(now, last_press) < 200: return None
    if key_a.value() == 1: last_press = now; return 'A'
    if key_b.value() == 1: last_press = now; return 'B'
    if key_c.value() == 1: last_press = now; return 'C'

```

```
    return None

print(": A=C5 B=E5 C=G5")

while True:
    key = read_key()
    if key:
        freq = NOTES[key]
        print(" %s: %d Hz" % (key, freq))
        play_tone(freq, 0.4)
    time.sleep_ms(20)
```

11.6 骰子+体感 (摇晃触发)

```
import protocol as p
from machine import Pin
import time
import urandom

key_a = Pin(10, Pin.IN, Pin.PULL_DOWN)
key_b = Pin(1, Pin.IN, Pin.PULL_DOWN)
key_c = Pin(3, Pin.IN, Pin.PULL_DOWN)

#
i2c = p.hal.i2c_map['ACC']
i2c.writeto_mem(0x18, 0x20, bytes([0x57]))
i2c.writeto_mem(0x18, 0x23, bytes([0x00]))

def read_accel():
    data = i2c.readfrom_mem(0x18, 0x28, 6)
    x = (data[1]<<8)|data[0]
    y = (data[3]<<8)|data[2]
    z = (data[5]<<8)|data[4]
    if x>=32768: x-=65536
    if y>=32768: y-=65536
    if z>=32768: z-=65536
    return (x,y,z)

#
# ... ()

#
shake_last = read_accel()
shake_cooldown = 0

def detect_shake(threshold=70000):
    global shake_last, shake_cooldown
    if shake_cooldown > 0:
        shake_cooldown -= 1
        return False
    x,y,z = read_accel()
    dx = abs(x-shake_last[0])
    dy = abs(y-shake_last[1])
    dz = abs(z-shake_last[2])
    shake_last = (x,y,z)
    if dx+dy+dz > threshold:
        shake_cooldown = 60
```

```

        return True
    return False

# A
while True:
    key = read_key() # read_key
    shaken = detect_shake()
    if (key == 'A' or shaken) and not rolling:
        rolling = True
        roll_dice() # roll_dice
        rolling = False
    time.sleep_ms(20)

```

第12章 常见问题与故障排除

12.1 启动问题

问题	可能原因	解决方法
串口无输出	波特率错误	设置为115200, 8N1
串口乱码	波特率不匹配	确认波特率115200
只显示"HT protocol"后卡住	main.py死循环	按Ctrl+C中断, 检查main.py
反复重启	main.py报错	查看错误信息, 修复代码
无法进入REPL	main.py阻塞	按Ctrl+C多次, 或重新烧录固件

12.2 点阵问题

问题	可能原因	解决方法
点阵全不亮	协控未初始化	调用 `p.check_hardware()` 或 `p.init_files()`
点阵全亮不灭	数据错误	发送 `p.uart_write(8, bytes([0,0,0,0]))`
单列不亮	位映射错误	确认使用高5位, 数据左移3位
图案上下颠倒	行序错误	bit7=顶部, bit3=底部, 确认 `1 << (7-y)`
图案左右颠倒	列序错误	字节0=最左列, 字节4=最右列
闪烁不稳定	发送过快	两次显示之间加 `time.sleep_ms(10)`

12.3 音频问题

问题	可能原因	解决方法
播放无声	文件不存在	确认文件名正确, `os.listdir()` 查看
播放无声	格式不支持	确认8位22050Hz WAV或原始PCM
声音卡顿	发送过快	每包之间加 `time.sleep_ms(5)`
声音过小	PCM幅值不够	放大波形幅值 (静音值128, 最大255)
爆音	波形突变	首尾加淡入淡出 (500采样渐变)
play()无反应	协控忙	等待当前播放结束, 或重新发送启动指令
录音无声	麦克风问题	检查麦克风连接, 或硬件故障
录音播放太快	采样率不匹配	原始PCM默认22050Hz

12.4 按键问题

问题	可能原因	解决方法
按键无反应	输入模式错误	使用 `Pin.IN, Pin.PULL_DOWN`, 按下=高电平
按键一直触发	消抖不足	增加消抖时间到200-300ms
按A触发B	引脚映射错误	A=GPIO10, B=GPIO1, C=GPIO3
按键失灵	硬件损坏	用 `pin.value()` 直接读取测试
组合键不工作	检测逻辑错误	同时检测多个按键的value

12.5 加速度计问题

问题	可能原因	解决方法
读取全0	未初始化	写 CTRL_REG1(0x20)=0x57
I2C通信失败	地址错误	地址0x18, 用 `i2c.scan()` 扫描
值不变化	轴未使能	CTRL_REG1的bit0-2设为1
值跳动大	数据率太高	降低到50Hz(0x47)
角度不准	未校准	平放记录基准值并减去
摇晃检测失灵	阈值太高	降低threshold到50000-70000

12.6 文件系统问题

问题	可能原因	解决方法
写入失败	空间不足	删除不需要的文件，`os.remove()`
文件损坏	写入中断	写入后确认，避免中途断电
无法删除	文件被占用	关闭所有打开的文件句柄
目录操作失败	目录非空	先删除目录内所有文件
中文文件名乱码	编码问题	使用英文文件名

12.7 内存问题

问题	可能原因	解决方法
MemoryError	内存不足	调用 `gc.collect()` 回收，减少大变量
程序变慢	内存碎片	定期 `gc.collect()`，避免频繁创建大对象
栈溢出	递归过深	减少递归层数，改用循环

12.8 固件恢复

如果设备完全无法启动，按以下步骤恢复：

1. **进入下载模式**：按住BOOT键，按一下RESET键，松开BOOT键 2. **擦除Flash**： ``bash esptool.py --port COM3 erase\_flash `` 3. ****MicroPython**** ``bash esptool.py --port COM3 --baud 460800 write\_flash -z 0x0 firmware.bin `` 4. **重启设备**：按RESET键

注意：通用MicroPython固件没有 `protocol` 模块，需要使用设备原厂固件才能控制点阵和音频。

附录A 内置音效文件清单

文件名	大小(字节)	声道	时长	用途推测
alert.dat	20782	1	0.94s	警示/提示音
broken.dat	44974	1	2.04s	错误/损坏提示（最长）
drop.dat	27694	1	1.25s	掉落/下落音效
du.dat	16322	2	0.37s	短促"嘟"音（双声道）
electric.dat	37486	1	1.70s	电流/电击音效
funny.dat	26542	1	1.20s	搞笑/滑稽音效
msg.dat	17686	1	0.80s	消息通知音

pass.dat	18478	1	0.84s	通过/成功音
record_start.dat	21428	1	0.97s	录音开始提示音
record_end.dat	17686	2	0.40s	录音结束提示音（双声
right.dat	20782	1	0.94s	正确/对的音（与alert同大小)
tech.dat	29422	1	1.33s	科技感音效
wrong.dat	24812	1	1.12s	错误/错的音

统一参数：

- 格式：WAV (PCM)
- 采样率：22050 Hz
- 位深：8位
- 音频格式：1 (PCM)

附录B 5×5 字模库

数字

```
DIGITS = {
  '0': [".XXX.", "X...X", "X...X", "X...X", ".XXX."],
  '1': [".X..", ".XX..", ".X..", ".X..", ".XXX."],
  '2': [".XXX.", "X...X", ".X..", ".X..", "XXXXX"],
  '3': ["XXXXX", ".X..", ".XXX.", ".X..", "XXXXX"],
  '4': ["X...X", "X...X", "XXXXX", "...X", "...X"],
  '5': ["XXXXX", "X...X", "XXXX.", "...X", "XXXX."],
  '6': [".XXX.", "X...X", "XXXX.", "X...X", ".XXX."],
  '7': ["XXXXX", "...X", "...X", ".X..", ".X..."],
  '8': [".XXX.", "X...X", ".XXX.", "X...X", ".XXX."],
  '9': [".XXX.", "X...X", ".XXXX", "...X", ".XXX."],
}
```

大写字母

```
LETTERS = {
  'A': [".XXX.", "X...X", "XXXXX", "X...X", "X...X"],
  'B': ["XXXX.", "X...X", "XXXX.", "X...X", "XXXX."],
  'C': [".XXXX", "X...X", "X...X", "X...X", ".XXXX"],
  'D': ["XXXX.", "X...X", "X...X", "X...X", "XXXX."],
  'E': ["XXXXX", "X...X", "XXXX.", "X...X", "XXXXX"],
  'F': ["XXXXX", "X...X", "XXXX.", "X...X", "X...X"],
  'G': [".XXXX", "X...X", "X..XX", "X...X", ".XXXX"],
  'H': ["X...X", "X...X", "XXXXX", "X...X", "X...X"],
  'I': [".XXX.", ".X..", ".X..", ".X..", ".XXX."],
  'J': ["....X", "...X", "...X", "X...X", ".XXX."],
  'K': ["X...X", "X..X.", "XXX..", "X..X.", "X...X"],
  'L': ["X...X", "X...X", "X...X", "X...X", "XXXXX"],
  'M': ["X...X", "XX.XX", "X.X.X", "X...X", "X...X"],
  'N': ["X...X", "XX.X", "X.X.X", "X..XX", "X...X"],
  'O': [".XXX.", "X...X", "X...X", "X...X", ".XXX."],
  'P': ["XXXX.", "X...X", "XXXX.", "X...X", "X...X"],
}
```

```

'Q': ["XXX.", "X...X", "X.X.X", "X..X.", "XX.X"],
'R': ["XXXX.", "X...X", "XXXX.", "X..X.", "X...X"],
'S': ["XXXX", "X...", "XXX.", "...X", "XXXX.",
'T': ["XXXXX", "...X.", "...X.", "...X.", "...X.",
'U': ["X...X", "X...X", "X...X", "X...X", "XXX."],
'V': ["X...X", "X...X", "X...X", ".X.X.", "...X."],
'W': ["X...X", "X...X", "X.X.X", "XX.XX", "X...X"],
'X': ["X...X", ".X.X.", "...X.", ".X.X.", "X...X"],
'Y': ["X...X", ".X.X.", "...X.", "...X.", "...X."],
'Z': ["XXXXX", "...X.", "...X.", ".X...", "XXXXX"],
}

```

符号

```

SYMBOLS = {
' ': [".....", ".....", ".....", ".....", "....."],
'!': ["..X..", ".X..", ".X..", ".....", ".X.."],
'?': ["XXX.", "X...X", ".X..", ".....", ".X.."],
'.': [".....", ".....", ".....", ".....", ".X.."],
',': [".....", ".....", ".....", ".X..", ".X.."],
'-': [".....", ".....", "XXXXX", ".....", "....."],
'+': [".....", ".X..", "XXXXX", ".X..", "....."],
'=': [".....", "XXXXX", ".....", "XXXXX", "....."],
'*': [".....", "X.X.X", ".XXX.", "X.X.X", "....."],
'/': ["...X", "...X.", ".X..", ".X..", "X..."],
'\\': ["X...", ".X...", ".X..", "...X.", "...X"],
'#': ["X.X.", "XXXXX", ".X.X.", "XXXXX", ".X.X."],
'%': ["XX..X", "XX.X.", "...X.", ".X.XX", "X..XX"],
'&': ["XX..", "X..X.", ".XX.", "X.X.X", ".X.X."],
'(': ["..X..", ".X..", "X...", ".X..", "...X."],
')': ["..X..", "...X.", "....X", "...X.", "...X."],
'[': ["XXXXX", "X...", "X...", "X...", "XXXXX"],
']': ["XXXXX", "...X", "....X", "....X", "XXXXX"],
'<': ["...X.", ".X..", ".X..", "...X.", "...X."],
'>': ["X...", ".X..", "...X.", "...X.", ".X..."],
'^': ["..X..", ".X.X.", "X...X", ".....", "....."],
'_': [".....", ".....", ".....", ".....", "XXXXX"],
'~': [".....", ".X.X.", "X.X.X", ".....", "....."],
'@': ["XXX.", "X.X.X", "XX.XX", "X...", ".XXXX"],
}

```

图案

```

PATTERNS = {
'heart': ["X.X.", "XXXXX", "XXXXX", ".XXX.", "...X."],
'smile': ["XXX.", "X.X.X", "XXXXX", "X.X.X", ".XXX."],
'frown': ["XXX.", "X.X.X", "XXXXX", "X.X.X", "X...X"],
'arrow_up': ["..X..", ".XXX.", "XXXXX", ".X..", "...X."],
'arrow_down': ["..X..", "...X.", "XXXXX", ".XXX.", "...X."],
'arrow_left': ["..X..", ".X...", "XXXXX", ".X...", "...X."],
'arrow_right': ["..X..", "...X.", "XXXXX", "...X.", "...X."],
'square': ["XXXXX", "X...X", "X...X", "X...X", "XXXXX"],
'circle': ["XXX.", "X...X", "X...X", "X...X", ".XXX."],
'star': ["..X..", ".XXX.", "XXXXX", ".X.X.", "X...X"],
'cross': ["..X..", "...X.", "XXXXX", "...X.", "...X."],
'x': ["X...X", ".X.X.", "...X.", ".X.X.", "X...X"],
}

```

```
'check': [". . . . .", ". . . . X", ". . . X .", ". X . X .", ". X . . ."],
'diamond': [". . X . .", ". X . X .", ". X . . X", ". X . X .", ". . X . ."],
'bell': [". . X . .", ". X . X .", ". X . X .", ". XXX .", ". . X . ."],
'note': [". . XX .", ". . X . X", ". . X . .", ". . X . .", "XX . . ."],
'fish': [". . . . .", ". XXXX", ". X . X .", "XXXXX", ". . . . ."],
'bird': [". X . . .", "XX . . .", "XXXXX", ". X . X .", ". . . . ."],
}
```

骰子点 (带边框)

```
DICE = {
  1: [ "XXXXX", "X . . X", "X . X . X", "X . . X", "XXXXX" ],
  2: [ "XXXXX", "XX . X", "X . . X", "X . . XX", "XXXXX" ],
  3: [ "XXXXX", "XX . X", "X . X . X", "X . . XX", "XXXXX" ],
  4: [ "XXXXX", "XX . XX", "X . . X", "XX . XX", "XXXXX" ],
  5: [ "XXXXX", "XX . XX", "X . X . X", "XX . XX", "XXXXX" ],
  6: [ "XXXXX", "XX . XX", "XX . XX", "XX . XX", "XXXXX" ],
}
```

附录C 硬件版本对比与兼容性指南

C.1 硬件版本总览

AlphaPi Zero

系列经历了多个硬件版本迭代，不同版本的核心模块、LED类型、音频格式存在差异。开发跨版本兼容代码时需特别注意。

版本	核心模块	LED类型	音频格式	协控芯片	典型固件日期
V1.03	`protocol` (frozen)	5×5点阵	.dat (WAV)	HT合泰MCU	2022-10-21
V1.40	`protocol` (frozen)	5×5点阵	.dat (WAV)	HT合泰MCU	2024-01-15
v1.05	`protocol` + `controlBoard` v1.1 (Zero Plus) `controlBoardZero Plus`	5×5点阵*	.dat (WAV)	HT合泰MCU	2024+
		NeoPixel RGB	.ima (ADPCM)	HT合泰MCU	2024+

*v1.05的LED类型待确认，protocol模块的uart_write接口应兼容点阵控制。

C.2 核心模块名差异

最关键的差异： V1.03使用拼错的模块名 `protocal`，V1.40及以后版本修正为 `protocol`。两者API完全一致。

```
V1.03: import protocal as p    #
V1.40: import protocol as p    #
v1.05: import protocol as p    #
v1.1:  import controlBoardZeroPlus as cb #
```

C.3 自动检测硬件版本

编写跨版本兼容代码时，使用以下模式自动检测并导入正确的模块：

```
#
try:
    import protocal as p
    HW_VERSION = 'V1.03'
except ImportError:
    try:
        import protocol as p
        HW_VERSION = 'V1.40/v1.05'
    except ImportError:
        try:
            import controlBoardZeroPlus as p
            HW_VERSION = 'v1.1 ZeroPlus'
        except ImportError:
            raise ImportError("")

print(":", HW_VERSION)
```

C.4 各版本API兼容性

C.4.1 V1.03 / V1.40 / v1.05 (protocol系列)

这三个版本的核心API完全一致，可共用同一份代码：

功能		函数	说明
点阵显示		<code>`p. uart_write(8, bytes([c0,c1,c2,c3,c4]))`</code>	5字节列数据
全屏点亮		<code>`p. led_fill()`</code>	无参数
进度条		<code>`p. progress(n)`</code>	n=0~255
播放音频		<code>`p. play('alert.dat')`</code>	.dat格式
录音		<code>`p. rec('filename')`</code>	原始PCM
硬件自检		<code>`p. check_hardware()`</code>	点阵闪烁
初始化		<code>`p. init_files()`</code>	文件系统初始化
板信息		<code>`p. board_info()`</code>	打印版本信息

C.4.2 v1.1 Zero Plus (controlBoardZeroPlus)

v1.1使用全新的 `controlBoardZeroPlus` 模块，API更丰富：

功能	函数	说明
LED显示	<code>`cb.led_show_bytes(bytes([c0..c4]))`</code>	兼容点阵接口
字符串显示	<code>`cb.showString("text")`</code>	内置字模，返回生成器
显示点阵列	<code>`cb.showPoints(points)`</code>	点阵列显示
清屏	<code>`cb.clearPoints()`</code>	清除显示
播放音频	<code>`cb.play('alert.ima')`</code>	.ima格式（ADPCM）
异步播放	<code>`cb.playAsync('alert.ima')`</code>	异步播放
停止播放	<code>`cb.stopPlay()`</code>	停止当前播放
录音	<code>`cb.rec('filename')`</code>	录音
加速度计	<code>`cb.GetAcceleration()`</code>	获取加速度
系统时间	<code>`cb.GetSysTime()`</code>	获取系统时间
WiFi连接	<code>`cb.connectWifi(ssid, password)`</code>	WiFi连接
开热点	<code>`cb.openHotspot(ssid, password)`</code>	AP模式
UDP通信	<code>`cb.send_udp(data)`</code> / <code>`cb.recv_udp()`</code>	UDP收发

v1.1扩展模块：

- `zeroPlusActuatorLedPad` - NeoPixel RGB LED驱动（WS2812）
- `zeroPlusSensorInfraredReceiver` - 红外接收传感器
- `actuator` - LED执行器
- `sensor` - 传感器
- `basic` - 基础工具（DataStruct等）

C.5 音频格式差异

版本	格式	编码	采样率	位深	扩展名
V1.03/V1.40/v1.05	WAV/PCM	未压缩	22050Hz	8位	.dat
v1.1 Zero Plus	ADPCM	压缩	-	-	.ima

跨版本音频播放：

```
def play_sound(filename_dat, filename_ima):
    if HW_VERSION == 'v1.1 ZeroPlus':
        cb.play(filename_ima)
    else:
        p.play(filename_dat)

play_sound('alert.dat', 'alert.ima')
```

C.6 LED类型差异

版本	LED类型	驱动方式	颜色	分辨率
V1.03/V1.40/v1.05	5×5点阵	HT协控行列扫描	单色	5×5
v1.1 Zero Plus	NeoPixel RGB	WS2812串行	RGB全彩	5×5*

*v1.1的LED数量和排列待确认，controlBoardZeroPlus的led_show_bytes接口提供了向后兼容。

跨版本LED显示：

```
def show_columns(cols):
    data = bytes(cols)
    if HW_VERSION == 'v1.1 ZeroPlus':
        cb.led_show_bytes(data)
    else:
        p.uart_write(8, data)

heart = [0x50, 0xF8, 0xF8, 0x70, 0x20]
show_columns(heart)
```

C.7 按键引脚一致性

好消息：所有版本的按键引脚定义完全一致！

按键	GPIO	输入模式	触发电平
A	GPIO10	下拉输入 (PULL_DOWN)	按下=高电平(1)
B	GPIO1	下拉输入 (PULL_DOWN)	按下=高电平(1)
C	GPIO3	下拉输入 (PULL_DOWN)	按下=高电平(1)

所有版本均可通过 `hal.key_map` 获取预定义的按键对象：

```
import hal
key_a = hal.key_map['A']
key_b = hal.key_map['B']
key_c = hal.key_map['C']
```

C.8 加速度计一致性

所有版本均使用 SC7A20 三轴加速度计，I2C地址0x18，SDA=GPIO6，SCL=GPIO7。

v1.1提供了更高级的封装：

```
# V1.03/V1.40/v1.05I2C
i2c = p.hal.i2c_map['ACC']
# ...

# v1.1API
x, y, z = cb.GetAcceleration()
```

C.9 产线测试工具

项目提供了一键产线测试工具 `production_test_tool.py`，支持自动检测硬件版本并上传对应测试固件：

```
-
- V1.03/V1.40/v1.05/v1.1
-
- LED + +
- /
-

run_test.bat

test_logs/
```

测试固件：

- `factory_test.py` - 适用于 V1.03/V1.40/v1.05 (protocol系列)
- `factory_test_zeroplus.py` - 适用于 v1.1 Zero Plus

C.10 版本检测代码（电脑端）

电脑端通过串口发送REPL命令检测硬件版本：

```
def detect_hardware(ser):
    # V1.03
    ser.write(b'import protocol\r\n')
    time.sleep(1)
    out = ser.read(ser.in_waiting).decode()
    if 'Error' not in out and 'Traceback' not in out:
        return 'V1.03', 'factory_test.py'

    # V1.40/v1.05
    ser.write(b'import protocol\r\n')
    time.sleep(1)
    out = ser.read(ser.in_waiting).decode()
    if 'Error' not in out and 'Traceback' not in out:
```

```
        return 'V1.40/v1.05', 'factory_test.py'

# v1.1 Zero Plus
ser.write(b'import controlBoardZeroPlus\r\n')
time.sleep(1)
out = ser.read(ser.in_waiting).decode()
if 'Error' not in out and 'Traceback' not in out:
    return 'v1.1 ZeroPlus', 'factory_test_zeroplus.py'

return 'UNKNOWN', 'factory_test.py'
```

C.11 跨版本开发最佳实践

1. 始终使用自动检测导入模式，不要硬编码模块名 2. 按键和加速度计代码可直接复用，所有版本一致 3. 音频和LED需做版本判断，不同版本格式/接口有差异 4. 优先使用高级API（如v1.1的showString、GetAcceleration），减少底层操作 5. 测试时覆盖所有版本，确保代码在每个版本上都能正常运行 6. 使用产线测试工具进行批量验证，确保硬件功能正常

C.12 常见兼容性问题

问题		原因	解决
`ImportError: no module named 'protocol`	在V1.40+上使用了V1.03的模块名		使用自动检测导入模式，或改用`protocol`
	在V1.03上使用了正确拼写		使用自动检测导入模式，或改用`protocol`
播放音频无声	v1.1使用.ima格式，代码传了.dat		根据版本选择对应格式的音频文件
LED显示异常	v1.1是NeoPixel RGB，不是点阵		使用`led_show_bytes`兼容接口，或使用NeoPixel专用API
`cb.showString` 无反应		showString返回生成器，需要迭代	`for _ in cb.showString("text"): pass`
WiFi功能不可用		仅v1.1支持WiFi	检查版本，V1.03/V1.40/v1.05无WiFi功能

附录D WS2812 RGB灯板扩展编程指南

D.1 硬件概述

AlphaPi Zero 主板提供 P1 扩展接口，可连接官方 RGB 灯板扩展模块。

参数	
规格	
LED型号	WS2812 (内置IC的全彩LED)
LED数量	14颗 (单面, 位于灯板反面)
排列方式	菱形/环形排列, 编号1-14
数据引脚	GPIO5
通信协议	单线串行 (WS2812协议)
颜色深度	24位 (8位R + 8位G + 8位B), 1677万色
供电	3.3V (P1接口提供)
接口	4针JST (VCC、GND、DIN、预留)

注意: 灯板正面为装饰面 (有编号丝印和贴片电阻), LED芯片均焊接在反面, 使用时反面向外。

D.2 P1 扩展接口引脚定义

引脚		功能	说明
1	VCC		3.3V电源输出
2	GND		地
3	DIN		WS2812数据输入 (GPIO5)
4	预留		未使用 (可用于级联DOUT或其他扩展)

D.3 基础驱动

AlphaPi Zero 的 MicroPython 固件内置 `neopixel` 模块, 使用 ESP32 的 RMT 外设实现精确的 WS2812 时序, 比软件模拟更可靠。

```
from machine import Pin
import neopixel
import time

# GPIO5 14LED
np = neopixel.NeoPixel(Pin(5), 14)

# LED0-13RGB
np[0] = (255, 0, 0) # 1
np[1] = (0, 255, 0) # 2
np[2] = (0, 0, 255) # 3

# writeLED
np.write()

# LED
np.fill((255, 255, 0)) #
np.write()

# LED
np.fill((0, 0, 0))
np.write()
```

重要:

- 颜色顺序是 ****(R, G, B)****, 每个分量0-255
- 设置颜色后必须调用 **np.write()** 才会刷新
- LED索引从0开始, 对应灯板编号1-14 (索引0=编号1)

D.4 RGBBoard 封装类

为了方便使用, 以下是一个封装好的 RGBBoard 类, 包含常用灯效:

```
from machine import Pin
import neopixel
import time

class RGBBoard:
    def __init__(self, pin=5, num=14, brightness=0.5):
        self.np = neopixel.NeoPixel(Pin(pin), num)
        self.num = num
        self._bri = brightness

    def bri(self, v=None):
        """ / 0.0~1.0 """
        if v is not None:
            self._bri = max(0, min(1, v))
        return self._bri

    def _c(self, r, g, b):
        """
        return (int(r*self._bri), int(g*self._bri), int(b*self._bri))
        """

    def set(self, i, r, g, b):
        """LED"""
        if 0 <= i < self.num:
            self.np[i] = self._c(r, g, b)

    def fill(self, r, g, b):
        """LED"""
        c = self._c(r, g, b)
        for i in range(self.num):
            self.np[i] = c

    def show(self):
        """
        self.np.write()
        """

    def clear(self):
        """LED"""
        self.fill(0, 0, 0)
        self.show()

    def rainbow(self, speed=15, rounds=3):
        """
        for _ in range(rounds):
            for j in range(256):
                for i in range(self.num):
                    rc = (i * 256 // self.num + j) & 255
                    if rc < 85: r,g,b = rc*3, 255-rc*3, 0
                    elif rc < 170: rc-=85; r,g,b = 255-rc*3, 0, rc*3
                """
```

```

        else: rc-=170; r,g,b = 0, rc*3, 255-rc*3
        self.mp[:] = self._c(r, g, b)
        self.show()
        time.sleep_ms(speed)

def breath(self, r, g, b, rounds=3, speed=12):
    """
    for _ in range(rounds):
        for v in range(0, 101, 2):
            self.bri(v/100); self.fill(r,g,b); self.show()
            time.sleep_ms(speed)
        for v in range(100, -1, -2):
            self.bri(v/100); self.fill(r,g,b); self.show()
            time.sleep_ms(speed)
    self.bri(0.5)

def chase(self, r, g, b, speed=50, rounds=3, tail=3):
    """
    for _ in range(rounds):
        for i in range(self.num + tail):
            self.clear()
            for t in range(tail):
                idx = i - t
                if 0 <= idx < self.num:
                    dim = 1 - t/tail
                    self.set(idx, int(r*dim), int(g*dim), int(b*dim))
            self.show()
            time.sleep_ms(speed)

def wipe(self, r, g, b, speed=40):
    """
    self.clear()
    for i in range(self.num):
        self.set(i, r, g, b)
        self.show()
        time.sleep_ms(speed)

def blink(self, r, g, b, times=5, speed=200):
    """
    for _ in range(times):
        self.fill(r, g, b); self.show()
        time.sleep_ms(speed)
        self.clear()
        time.sleep_ms(speed)

def meter(self, value, r, g, b, max_val=100):
    """/value 0~max_val"""
    self.clear()
    lit = int(value / max_val * self.num)
    for i in range(lit):
        self.set(i, r, g, b)
    self.show()

def gradient(self, r1, g1, b1, r2, g2, b2):
    """
    for i in range(self.num):
        t = i / (self.num - 1)

```

```
        r = int(r1 + (r2-r1)*t)
        g = int(g1 + (g2-g1)*t)
        b = int(b1 + (b2-b1)*t)
        self.set(i, r, g, b)
    self.show()
```

D.5 颜色常量

```
# RGB
RED      = (255, 0, 0)
GREEN    = (0, 255, 0)
BLUE     = (0, 0, 255)
YELLOW   = (255, 255, 0)
CYAN     = (0, 255, 255)
MAGENTA  = (255, 0, 255)
WHITE    = (255, 255, 255)
ORANGE   = (255, 165, 0)
PURPLE   = (128, 0, 128)
PINK     = (255, 192, 203)
```

D.6 配合按键的交互示例

```
# ABC
from machine import Pin
import time
import urandom
from ws2812_board import RGBBoard #

led = RGBBoard(pin=5, num=14, brightness=0.4)
key_a = Pin(10, Pin.IN, Pin.PULL_DOWN)
key_b = Pin(1, Pin.IN, Pin.PULL_DOWN)
key_c = Pin(3, Pin.IN, Pin.PULL_DOWN)

effects = ['rainbow', 'breath', 'chase', 'wipe', 'gradient']
effect_idx = 0
hue = 0

last_press = 0
def read_key():
    global last_press
    now = time.ticks_ms()
    if time.ticks_diff(now, last_press) < 200:
        return None
    if key_a.value(): last_press = now; return 'A'
    if key_b.value(): last_press = now; return 'B'
    if key_c.value(): last_press = now; return 'C'
    return None

print("A= B+= C+=")

while True:
    key = read_key()
    if key == 'A':
        effect_idx = (effect_idx + 1) % len(effects)
        print(":", effects[effect_idx])
    elif key == 'B':
```

```
b = led.bri() + 0.1
if b > 1.0: b = 0.1
led.bri(b)
print(": %.0f%%" % (b*100))
elif key == 'C':
    hue = (hue + 32) % 256
    print(":", hue)

#
if effects[effect_idx] == 'rainbow':
    led.rainbow(speed=10, rounds=1)
elif effects[effect_idx] == 'breath':
    led.breath(0, 255, 255, rounds=1, speed=10)
elif effects[effect_idx] == 'chase':
    led.chase(255, 0, 0, speed=40, rounds=1, tail=4)
elif effects[effect_idx] == 'wipe':
    led.wipe(0, 255, 0, speed=50)
    time.sleep(0.5)
    led.clear()
elif effects[effect_idx] == 'gradient':
    led.gradient(255, 0, 0, 0, 0, 255)
    time.sleep(1)
    led.clear()

time.sleep_ms(10)
```

D.7 配合加速度计的水平仪示例

```
# LED
import protocol as p
import time
import math
from ws2812_board import RGBBoard

led = RGBBoard(pin=5, num=14, brightness=0.5)
i2c = p.hal.i2c_map['ACC']
i2c.writeto_mem(0x18, 0x20, bytes([0x57]))
i2c.writeto_mem(0x18, 0x23, bytes([0x00]))

def read_accel():
    data = i2c.readfrom_mem(0x18, 0x28, 6)
    x = (data[1]<<8)|data[0]
    y = (data[3]<<8)|data[2]
    z = (data[5]<<8)|data[4]
    if x>=32768: x-=65536
    if y>=32768: y-=65536
    if z>=32768: z-=65536
    return (x/1000, y/1000, z/1000) # g

print("")

while True:
    xg, yg, zg = read_accel()

    #
    tilt = math.sqrt(xg*xg + yg*yg)
```

```
if tilt < 0.1:
    #
    led.fill(0, 255, 0)
else:
    # LED
    angle = math.atan2(yg, xg) #
    # LED0-13
    idx = int((angle + math.pi) / (2*math.pi) * 14) % 14

    #
    r = int(min(255, tilt * 500))
    g = int(max(0, 255 - tilt * 500))
    led.clear()
    led.set(idx, r, g, 0)
    #
    led.set((idx+1)%14, r//2, g//2, 0)
    led.set((idx-1)%14, r//2, g//2, 0)

led.show()
time.sleep_ms(50)
```

D.8 配合麦克风的音量指示器示例

```
# 14LED
import protocol as p
import time
from ws2812_board import RGBBoard

led = RGBBoard(pin=5, num=14, brightness=0.5)

#
# FFT

def get_volume():
    """
    #
    return 50 #

print("LED")

while True:
    vol = get_volume() # 0-100

    #
    if vol < 40:
        r, g, b = 0, 255, 0
    elif vol < 70:
        r, g, b = 255, 255, 0
    else:
        r, g, b = 255, 0, 0

    led.meter(vol, r, g, b, max_val=100)
    time.sleep_ms(50)
```

D.9 应用场景汇总

应用	用到的硬件	说明
音乐律动灯	麦克风 + WS2812	采集环境声音，LED随节奏/音量跳动
水平仪/倾角仪	加速度计 + WS2812	倾斜时对应方向LED亮，颜色表示角度
摇骰子	加速度计/按键 + WS2812	摇动触发，LED滚动后显示点数（颜色/
倒计时器	按键 + WS2812 + 蜂鸣器	LED数量显示剩余时间，颜色绿→黄→红
音量指示器	麦克风 + WS2812	实时显示环境音量，超阈值变红闪烁
氛围灯	按键 + WS2812	A切换灯效（彩虹/呼吸/追逐/纯色），B调亮度，C调色相
反应速度测试	按键 + WS2812	LED随机亮某颗，测按键反应时间
打地鼠游戏	按键 + WS2812	随机位置LED亮，按对应按键打掉，计分
状态指示灯	WiFi + WS2812	不同颜色表示WiFi状态（蓝=连接中、绿=已连、红=失败）
秒表/计时器	按键 + WS2812 + 点阵	A开始/B暂停/C结束，LED显示进度，点
报警提醒器	加速度计 + WS2812 + 蜂鸣器	检测震动/移动时红色闪烁+蜂鸣，防盗/
方向指示	加速度计 + WS2812	倾斜时对应方向LED亮，简易方向指示
WiFi天气灯	WiFi + WS2812	联网获取天气，颜色表示天气（蓝=晴、灰=阴、闪烁=下雨）

D.10 常见问题

问题	原因	解决
LED全不亮	数据引脚错误	确认使用GPIO5，P1接口第3针
LED全不亮	插头方向反了	4针JST插头反过来插，确认VCC/GND不
颜色不对（红变绿等）	颜色顺序错误	neopixel使用(R,G,B)顺序，确认参数顺
只有部分LED亮	LED数量设置错误	确认num=14，灯板共14颗LED
LED闪烁不稳定	供电不足	降低亮度（brightness<0.5），或使用外部5V供电
正面不亮	LED在反面	灯板LED均焊接在反面，正面是装饰面
某颗LED不亮	该LED损坏	WS2812级联，一颗损坏会导致后面全不
调用write()后无反应	未设置颜色	先设置np[i]=(r,g,b)再调用write()
亮度太暗	全局亮度太低	提高brightness参数，或直接设置255值
程序运行时LED冻结	阻塞操作	灯效函数中有sleep，长任务应在独立协程中运行

D.11 级联扩展

WS2812 支持级联，P1 接口第4针（预留）可引出 DOUT 信号连接更多 WS2812 灯条：

```
# 30LED14 + 16
np = neopixel.NeoPixel(Pin(5), 30)

# LED0-13
for i in range(14):
    np[i] = (255, 0, 0)

# 14-29
for i in range(14, 30):
    np[i] = (0, 0, 255)

np.write()
```

注意：级联时供电需求增加，建议外接5V电源，不要全部从主板3.3V取电。

手册完